Benchmarking Community Detection Algorithms with Overlaps and Outliers

by

Ryan DeWolfe

Bachelor of Science, The University of British Columbia, 2024

A thesis

presented to Ryerson University

in partial fulfillment of the

requirements for the degree of

Master of Science (M.Sc.)

in the program of

Applied Mathematics

Toronto, Ontario, Canada, 2026

©Ryan DeWolfe, 2026

AUTHOR'S DECLARATION FOR ELECTRONIC SUBMISSION OF A THESIS

I hereby declare that I am the sole author of this thesis. This is a true copy of the thesis, including any required final revisions, as accepted by my examiners.

I authorize Ryerson University to lend this thesis to other institutions or individuals for the purpose of scholarly research.

I further authorize Ryerson University to reproduce this thesis by photocopying or by other means, in total or in part, at the request of other institutions or individuals for the purpose of scholarly research.

I understand that my thesis may be made electronically available to the public.

Benchmarking Community Detection Algorithms with Overlaps and Outliers

Master of Science (M.Sc.), 2026

Ryan DeWolfe

Applied Mathematics

Ryerson University

Abstract

Clustering (or community detection) is one of the fundamental problems in unsupervised machine learning. For graph data, community detection algorithms look for subsets of vertices that are densely connected to each other and sparsely connected to the rest of the graph. The most popular community detection algorithms tend to insist each vertex belong to exactly one community, although recent extensions allow for overlaps (vertices belonging to multiple communities) and outliers (vertices belonging to no communities). This thesis develops tools for extrinsic benchmarking of community detection algorithms that with overlaps and outliers; first we propose an extension of the Artificial Benchmark for Community Detection to create synthetic graphs with known community structure, then we propose a clustering similarity measure that can explicitly handle overlaps and outliers. We conclude with an example of using the new tools to compare several community detection methods, and open the door for measuring further improvements.

Acknowledgements

Put acknowledgments here.

Table of Contents

Declaration ii

Abstract iii

Acknowledgements iv

List of Tables vii

List of Figures viii

List of Appendices xiv

1 Introduction **1**

2 The Artificial Benchmark for Community Detection with Overlaps and Outliers **5**

2.1 The Model 7

2.1.1 The Big Picture 7

2.1.2 Notation 7

2.1.3 The Configuration Model 9

2.1.4 Parameters 9

2.1.5 The Generation Algorithm 10

2.2 Comparing the Model to Real Data 18

2.2.1 Effects of the Reference Layer 20

2.2.2 Additional properties of the model. 25

3 A Pragmatic Method for Comparing Clusterings with Overlaps and Outliers **31**

3.1 Existing Options 32

3.1.1 Omega Index 32

3.1.2 Overlapping Normalized Mutual Information 33

3.1.3 Element Centric Clustering Similarity 34

3.1.4 The Problem of Matching 36

3.2 The Proposed Measure 36

3.3 Properties 38

3.4 Experiments 47

3.4.1 Intuitive Behaviour 47

3.4.2 Graph Aware Clustering Comparison 49

4 Benchmarking Community Detection Algorithms **54**

4.1 Community Detection Algorithms 54

4.1.1 Egosplit + CAS 54

4.1.2 LC + HSLC 56

4.1.3 OSLOM 58

4.2 Synthetic Data 59

4.3 Real Data 62

5 Conclusion **65**

Appendices **66**

References **81**

List of Tables

2.1 Parameters of the ABCD+o^2 model. 10

2.2 The parameters used for generating ABCD+o^2 graphs based on the DBLP, Amazon, and YouTube datasets. 19

2.3 Measured ρ in both the real networks and the corresponding ABCD+o^2 graphs. We generated 10 ABCD+o^2 graphs for each dimension d of the reference layer, and found that across all graphs and dimensions, the standard error, to two decimal places, was 0. 25

List of Figures

1.1 An example of a graph with an intuitive clustering (represented by vertex colour) with overlap and an outlier (the center grey vertex). 2

2.1 A big picture example of generating an ABCD+o^2 graph. In panel A, we assign nodes to communities; each colour represents a community and black nodes represent outliers. Note that some nodes belong to multiple communities. In panel B, we assign degrees to nodes and split each degree into the background degree (black stubs) and community degree (coloured stubs). In panel C, we build each graph using the configuration model. Finally in panel D, we perform edge swaps to resolve self-loops and parallel edges. 8

2.2 Example of building the community C_3 with $\hat{s}_3 = 40$ and $\eta = 1.75$ using a 2-dimensional reference layer. First, in panel A, we select the seed c_3 as the vector furthest from the origin that does not have a primary community. Next (panel B), we build the primary community $\hat{C}_3$ as node of the seed and the nodes of the seed's $\hat{s}_3 - 1$ nearest neighbours that do not yet have a primary community. Finally, in panel C, we expand create C_3 by expanding $\hat{C}_3$ by a factor of η (from 40 to $40 \cdot 1.75 = 70$ nodes) by taking the nearest neighbours to the primary center of mass. 13

2.3 Distribution of community sizes for real-world networks and their synthetic counterparts: DBLP in panel A, Amazon in panel B, and Youtube in panel C. The y-axis shows the empirical complementary cumulative distribution function, computed as $y = 1 - |\{C_i : |C_i| < x\}| \,/\, |\{C_i\}|$. 21

2.4 Distribution of the number of communities per node in each real-world network and their synthetic counterparts: DBLP in panel A, Amazon in panel B, and Youtube in panel C. The y-axis shows the empirical complementary cumulative distribution function, computed as $y = 1 - |\{v_i : |\{C_j : v_i \in C_j\} < x\}| \,/\, |V|$. 22

2.5 Distribution of the size of overlaps in real graphs and their synthetic counterparts. Each row corresponds to a real graph, panels A,B,C for DBLP, panels D,E,F for Amazon, and panels G,H,I for Youtube. The left (panels A,D,G), center (panels B,E,H) and right (panels C,F,I) columns correspond to overlaps of two, three, and four communities respectively. The y-axis shows the empirical complementary cumulative distribution function of non-empty overlaps. For example, the center column of figures corresponds to three overlapping communities and is computed as $y = 1 - |\{\{C_i, C_j, C_k\} : |C_i \cap C_j \cap C_k| < x\}| \,/\, |\{\{C_i, C_j, C_k\} : |C_i \cap C_j \cap C_k| \neq \emptyset|$. 24

2.6 Community overlap densities compared to individual community densities for various values of ρ in Youtube-like graphs (panel A) and DBLP-like graphs (panel B) with $d = 8$. For 9 ABCD+o^2 graphs with ρ evenly spaced in $[-0.5, 0.5]$, we compute the empirical value $\hat{\rho}$ and the distribution of density for overlapping communities. We plot $\hat{\rho}$ against the distribution of the density for the intersection and each community individually. The line corresponds to the median, and the shaded region to the 25th to 75th quantile. For computational reasons, we only consider overlaps of size at least 25, and to ensure we are not simply measuring the density of the smaller community, the size of the overlap must be at most half the size of the smaller community. 26

2.7 Distribution of the density for pairs of communities with non-zero overlap in real graphs and their synthetic counterparts; DBLP in panel A, Amazon in panel B, and Youtube in panel C. The shaded region corresponds to the 25th to 75th quartiles, and the whiskers correspond to the maximum and minimum values. We restrict to overlaps of size at least 10 due to the discrete and unpredictable nature of small communities. 28

2.8 Five largest internal edge fractions, sorted in descending order, grouped by the number of communities. Panel A corresponds to the DBLP graphs, and panel B to the synthetic counterpart with reference layer dimension $d = 8$. A pattern is consistent between DBLP and ABCD+o^2, namely, that for nodes belonging to ℓ communities there are ℓ large, but not equal, internal edge fraction values. 30

3.1 Recreation of the experiment in Figure 2 of [27] with F^*_{wo}. Each column represents one scenario, including a visual of the scenario, the intuitive behaviour of a similarity measure in this scenario, and the actual behaviour of several measures in each row respectively. Each line is the average of 100 simulations and the shaded region (usually too small to appear) covers plus or minus one standard deviation. In each scenario, the proposed F^*_{wo} measure matches our intuition. 49

3.2 Comparing comparison measure behaviour on intuitive scenarios when overlaps or outliers are present. The top row is the scenario with overlapping clusters, and the second row has outliers; the columns, from left to right, show a visual of the scenario, the intuitive behaviour, and the actual behaviour of each method. 52

3.3 An experiment comparing graph aware clustering comparison, similar to that of Figure 5 from [54]. We use three resolutions (0.5, 1.0, and 3.75 shown in red, green, and yellow respectively) of the Leiden clustering algorithm on ABCD graphs with 2000 vertices and show, from left to right, the similarity of each detected vertex clustering to the ground truth vertex clusters, the similarity of the detected induced edge clustering to the ground truth induced edge clusters, and the number of clusters The solid line represents the average over 5 samples and the shaded region covers plus or minus one standard deviation. 53

4.1 Evaluating the performance of each algorithm with respect to variable ξ and η. 60

4.2 Evaluating the performance of each clustering algorithm with fixed $\eta = 1.5$ (left column) or $\xi = 0.3$ (right column). 61

4.3 Ability of each community detection algorithm to match the full ground truth clustering on real graph datasets. 63

4.4 Ability of each clustering algorithm to find the top 5000 ground truth communities on real graph datasets. 64

A.1 The density of overlaps compared to the empirical ρ on YouTube-like ABCD+o^2 graphs with reference layer dimensions $d = 2$ (panel A) and $d = 64$ (panel B). 67

A.2 The density of overlaps compared to the empirical ρ on DBLP-like ABCD+o^2 graphs with reference layer dimensions $d = 2$ (panel A) and $d = 64$ (panel B). 67

A.3 The density of overlaps compared to the empirical ρ on Amazon-like ABCD+o^2 graphs with reference layer dimensions $d = 2$ (panel A), $d = 8$ (panel B) and $d = 64$ (panel C). 68

A.4 The top 5 IEF values in decreasing order, grouped by the number of communities the node belongs to, for DBLP and DBLP-like ABCD+o^{2}graphs. Panel A corresponds to the real graph, and panels B, C, D correspond to the synthetic counterparts with reference layer dimension $d = 2$, $d = 8$, and $d = 64$ respectively. Panels A and C correspond to panels A and B of Figure 2.8, but are repeated here for easy comparison with the other reference layer dimensions. 69

A.5 The top 5 IEF values in decreasing order, grouped by the number of communities the node belongs to, for Amazon and Amazon-like ABCD+o^{2}graphs. Panel A corresponds to the real graph, and panels B, C, D correspond to the synthetic counterparts with reference layer dimension $d = 2$, $d = 8$, and $d = 64$ respectively 70

A.6 The top 5 IEF values in decreasing order, grouped by the number of communities the node belongs to, for YouTube and YouTube-like ABCD+o^{2}graphs. Panel A corresponds to the real graph, and panels B, C, D correspond to the synthetic counterparts with reference layer dimension $d = 2$, $d = 8$, and $d = 64$ respectively 71

List of Appendices

A Additional ABCD+o^2 Figures **66**

A.1 Density of Intersection Figures 66

A.2 Internal Edge Fraction Figures 68

B Python Code for Comparing Clusterings **72**

Chapter 1

Introduction

Clustering – or in the context of graph data, community detection – is one of the fundamental problems in unsupervised machine learning [31]. A clustering algorithm finds subsets of the data that intuitively "belong together", a notoriously ill-defined idea that is widely used for preprocessing, exploration, and prediction [**RD:** *citations*]. For graph data, the typical idea of a good community is a subset of the vertices that are densely connected to each other while sparsely connected to the rest of the graph. There have been a variety of attempts to exactly define a community, or score the quality of a community, and an even more algorithms to detect the various definitions of community [25].

The most popular and well studied ideas of community focus on partitions [28, 12, 42], a clustering where every vertex belongs to exactly one cluster. However, especially in graph data, partitions are not always sufficient to describe the intuitive clusters (Figure 1.1). In particular, we extend the notion of a clustering to allow for outliers (vertices may belong to no clusters) and overlaps (vertices may belong to multiple communities). There are some community detection algorithms for this regime [44, 20], but evaluating and comparing these algorithms can be difficult.

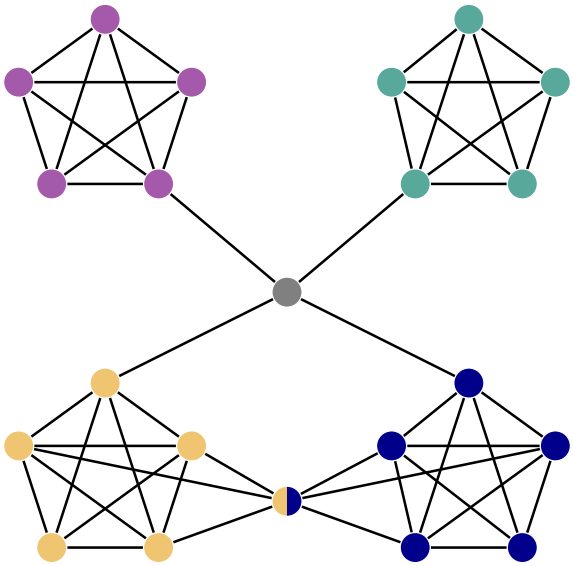


Figure 1.1: An example of a graph with an intuitive clustering (represented by vertex colour) with overlap and an outlier (the center grey vertex).

This thesis develops tools for the extrinsic benchmarking of community detection algorithms that allow for outliers and overlaps. An extrinsic benchmark evaluates the ability of a community detection algorithm to find a ground truth clustering that is believed to be correct. The first component to an extrinsic benchmark is the data. In real networks, the ground truth clustering is often determined from meta-data that is not given to the detection algorithm but is suspected to produce communities [46]. However, creating these ground truth clustering is difficult, and it is not always clear that the ground truth communities actually exist in the data [65]. Thus, we use synthetic data that is generated with a known community structure [42, 40, 35, 38]. In Chapter 2, we propose ABCD+o^2, an extension of the Artificial Benchmark for Community Detection with Outliers (ABCD+o) model [38, 35] to allow for overlapping communities. The second component is a method to compare the similarity of the detected clustering to the ground truth clustering. Many methods exist for partitions [33, 39, 61], and some

have extended for overlaps [19, 41, 27], but to the best of our knowledge none have explicitly considered outliers. Chapter 3 develops a new method based on a set-matching framework [56] and includes an explicit outlier comparison term. Finally, Chapter 4 briefly reviews three community detection algorithms and uses the newly developed tools to evaluate their performance.

The bulk of this thesis is based on the following papers:

- "The Artificial Benchmark for Community Detection with Outliers and Overlapping Communities (ABCD+o^2)" [7] has been accepted for publication in the Journal of Complex Networks. A preliminary version [8] was published in the Proceedings of the 20th Workshop on Modelling and Mining Networks (WAW2025). The authors contributed equally to the results.

- "A Pragmatic Method for Comparing Clusterings with Overlaps and Outliers" [**RD:** *is under review somewhere.*] Ryan DeWolfe was the lead author, responsible for the idea, implementation, experiments, and initial draft.

It also uses community detection algorithms developed during the program and presented in the following papers:

- "Improving Community Detection via Community Association Strength Scores" [6] published in the Proceedings of the 20th Workshop on Modelling and Mining Networks (WAW2025). The authors contributed equally to the results.

- "Hierarchical Single-Linkage Clustering for Community Detection with Overlaps and Outliers" [20] published in the Proceedings of the 14th International Conference on Complex Networks and their Applications (COMPLEX NETWORKS 2025). Ryan DeWolfe is the sole author.

Finally, written during the program but not mentioned in the thesis are the following papers:

- "Detecting Patterns of Interaction in Temporal Hypergraphs via Edge Clustering" [23] published in the Proceedings of the 17th International Conference on Advances in Social Networks Analysis and Mining (ASONAM2025).
- "Leveraging Non-linear Dimension Reduction and Random Walk Co-occurrence for Node Embedding" [21] posted to arXiv.

Chapter 2

The Artificial Benchmark for Community Detection with Overlaps and Outliers

To support the development and analysis of overlapping community detection algorithms, a large and diverse catalogue of networks with ground-truth communities is required for testing, tuning and training. Unfortunately, there are few such datasets with communities identified and labelled. As a result, there is a need for synthetic random graph models with known community structure that resemble real-world networks. The popular LFR (Lancichinetti, Fortunato, and Radicchi) model [42, 40] generates networks with communities and, at the same time, allows for heterogeneity in the distributions of both node degrees and of community sizes. Due to this structural freedom, the LFR model became a standard and extensively used model for generating artificial networks with ground-truth communities. A model similar to LFR, the Artificial Benchmark for Community Detection (ABCD) [35], was recently introduced in 2021.

Undirected variants of LFR and ABCD produce graphs with comparable properties, but ABCD is faster than LFR and can be easily tuned to allow the user to make a smooth

transition between the two extremes: pure (disjoint) communities and a random graph with no community structure. Moreover, ABCD is easier to analyze theoretically [34, 9]. Finally, the building blocks in the model are flexible and may be adjusted to satisfy different needs. The ABCD model has been generalized to include outliers (ABCD+o) [38] and separately generalized to generate hypergraphs [37]. Accordingly, the ABCD model is gaining recognition as a benchmark for community detection algorithms. For example, in [4], the authors use both ABCD and LFR graphs to compare 30 community detection algorithms and highlight that *"being directly comparable to LFR, ABCD offers additional benefits, including higher scalability and better control for adjusting an analogous mixing parameter."*

This chapter introduces the ABCD+o^2 model: a further generalization of the ABCD+o model that allows for overlapping communities. The LFR model has been similarly extended [40], but in their model the nodes are assigned to communities based on the construction of a random bipartite graph between nodes and communities resulting in a small amount of overlap between almost every pair of communities and rarely any pair of communities with a large overlap. In ABCD+o^2, we instead generate overlapping communities based on a hidden, low-dimensional geometric layer which tends to yield fewer and larger overlaps. This geometric approach to community structure is justified by the fact that latent geometric spaces are believed to shape complex networks (e.g., social media networks shaped by users' opinions, education, knowledge, interests, etc.). These latent spaces have been successfully employed for many years to model and explain network properties such as self-similarity [59], homophily and aversion [32]. For more details, we direct the reader to the survey [13] or the book [58]. The ancillary benefits of the ABCD model (an intuitive noise parameter, a fast implementation, and theoretical analysis) are still present.

2.1 The Model

2.1.1 The Big Picture

The ABCD+o^2 model generates a random graph on n nodes with a degree sequence and a community size sequence that (by default) follow a truncated power law distribution with exponent γ and β respectively. The model samples a graph in four phases, which are visualized in Figure 2.1. In the first phase, we assign non-outlier nodes to communities, making use of a d-dimension hidden reference layer whose geometry is used to guide the community overlap. Next, we sample and assign degree to nodes. In phase 3, the degree of each node is split so that (on average) the proportion in the background graph is ξ and the remaining degree is split evenly among the communities to which the node belongs. We sample the background graph and each community graph using the configuration model. Finally, we perform any necessary rewiring to remove loops and multi-edges so that the final graph is simple.

2.1.2 Notation

For $n \in \mathbb{N} := \{1, 2, \ldots\}$, let $[n]$ denote the set consisting of the first n natural numbers, that is, $[n] := \{1, 2, \ldots, n\}$. For a random variable X, write $\mathbb{P}(X = k)$ for the probability that $X = k$ and $\mathbb{E}[X]$ for the expected value of X. For a distribution $\mathcal{D}$, write $X \sim \mathcal{D}$ to mean X is sampled from the distribution $\mathcal{D}$. Any real number $x \in \mathbb{R}$ can be written as $x = a + b$ with $a \in \mathbb{Z}$ and $b \in [0, 1)$. Define a random variable

$$\lfloor x \rceil := \begin{cases} a & \text{with probability } 1 - b, \text{ and} \\ a + 1 & \text{with probability } b\,. \end{cases}$$

Note that $\mathbb{E}\left[\lfloor x \rceil\right] = a(1 - b) + (a + 1)b = x$.

For given parameters $\gamma \in (0, \infty)$, $\delta, \Delta \in \mathbb{N}$ with $\delta \leq \Delta$, define a truncated power-law

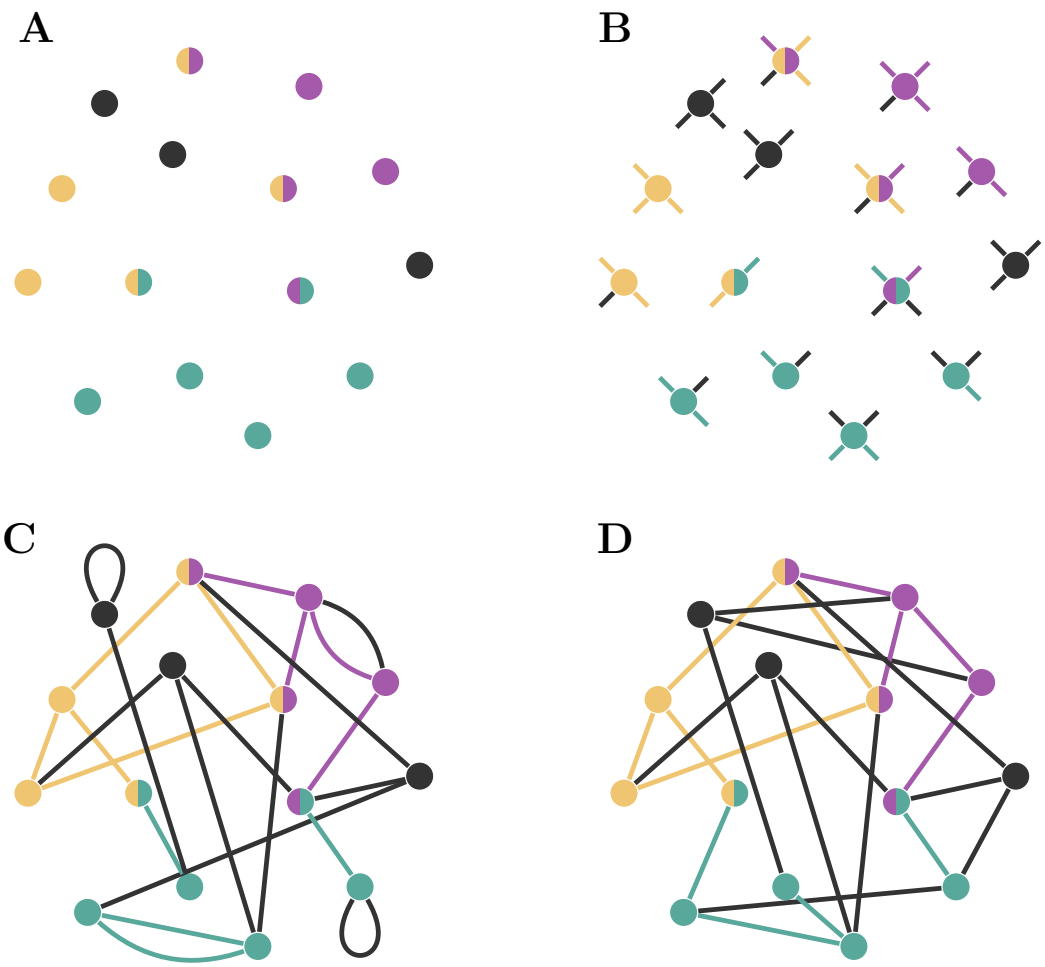


Figure 2.1: A big picture example of generating an ABCD+o^2 graph. In panel A, we assign nodes to communities; each colour represents a community and black nodes represent outliers. Note that some nodes belong to multiple communities. In panel B, we assign degrees to nodes and split each degree into the background degree (black stubs) and community degree (coloured stubs). In panel C, we build each graph using the configuration model. Finally in panel D, we perform edge swaps to resolve self-loops and parallel edges.

distribution $\mathcal{P}(\gamma, \delta, \Delta)$ as follows. For $X \sim \mathcal{P}(\gamma, \delta, \Delta)$ and for $k \in \mathbb{N}$ with $\delta \leq k \leq \Delta$,

$$\mathbb{P}(X = k) = \frac{\int_k^{k+1} x^{-\gamma}\, dx}{\int_\delta^{\Delta+1} x^{-\gamma}\, dx}\,. \tag{2.1}$$

2.1.3 The Configuration Model

The configuration model is an important ingredient of all variants of the ABCD models. Suppose that our goal is to create a graph on n nodes with a given degree sequence $\mathbf{d}$, where $\mathbf{d} = (d_i, i \in [n])$ is a sequence of non-negative integers such that $\sum_{i\in[n]} d_i$ is even. The configuration model CM($\mathbf{d}$), first introduced by Bollobás [14], samples a multi-graph with the given degree sequence $\mathbf{d}$. The model first labels n nodes as $(v_i, i \in [n])$ and endowing each v_i with d_i half-edges. It then iteratively choose two unpaired half-edges uniformly at random from the set of pairs of remaining half-edges, and pair them together to form an edge. This process yields a multi-graph $G_n \sim \text{CM}(\mathbf{d})$ on n nodes, where G_n is allowed self-loops and multi-edges. See [10, 62, 63] for related models and results.

2.1.4 Parameters

Table 2.1 summarizes the parameters that govern the ABCD+o^2 model. Note that the ranges for parameters γ and β are suggestions chosen according to experimental values commonly observed in complex networks [5, 51]. Furthermore, although power-law distributions are a sensible default option, the implementation allows the user to pass a custom degree or primary community size sequence (defined in phase 1 of section 2.1.5).

Parameter	Range	Description
n	$\mathbb{N}$	Number of nodes
s_0	$\mathbb{N}$	Number of outliers
η	$[1, \infty)$	Average number of communities a non-outlier node is part of
d	$\mathbb{N}$	Dimension of the reference layer
ρ	$[-1, 1]$	Pearson correlation between the degree of nodes and the number of communities they belong to
γ	$(2, 3)$	Exponent for power-law degree distribution
δ	$\mathbb{N}$	Min degree
Δ	$\{\delta, \delta+1, \ldots\}$	Max degree
β	$(1, 2)$	Exponent for power-law community size distribution
s	$\{\delta+1, \delta+2, \ldots\}$	Min community size
S	$\{s, s+1, \ldots\}$	Max community
ξ	$[0, 1]$	Level of noise

Table 2.1: Parameters of the ABCD+o^2 model.

2.1.5 The Generation Algorithm

Phase 1: assigning communities.

This phase is the essential contribution of the ABCD+o^2 extension. We label the nodes $v_1, \ldots, v_n$ such that nodes $v_{\hat{n}+1}, \ldots, v_n$ are the outliers, with $\hat{n} = n - s_0$.

The proposed method embeds nodes in a d-dimensional latent space in which we first create non-overlapping *primary* communities, and then expand each community by (in expectation) a factor of η to create overlap. The user may pass a sequence $(\hat{s}_j, j \in [L])$ of primary community sizes that we relabel so $\hat{s}_1 \geq \cdots \geq \hat{s}_L$. Otherwise, we sample the primary community size sequence from a truncated power-law $\mathcal{P}\left(\beta, \hat{s}, \hat{S}\right)$, where $\hat{s} = \lceil s/\eta \rceil$ and $\hat{S} = \lfloor S/\eta \rfloor$. We sample $\hat{s}_j \sim \mathcal{P}\left(\beta, \hat{s}, \hat{S}\right)$ independently until the sum of samples is at least $\hat{n}$. Since the sum of primary community sizes must equal $\hat{n}$, if the sum is $\hat{n} + a$ for some $a > 0$ then we fix the sequence as follows. If the last sample $\hat{s}_{\max j}$ has size at least $a + \hat{s}$, we reduce it by a. Otherwise, we delete the last sample and select $\hat{s}_{\max j} - a$ other samples to increase by 1. Let L be the random variable counting the

number of communities, and relabel $(\hat{s}_j, j \in [L])$ so that $\hat{s}_1 \geq \cdots \geq \hat{s}_L$.

At this point, let us pause to ensure the intuition for η matches its use. A primary community with size $\hat{s}_j$ will expand to a final size of $s_j = \lfloor \eta \hat{s}_j \rceil$. Thus, after expanding each community, the average number of communities per non-outlier node (in expectation) is

$$\mathbb{E}\left[\frac{\sum_{j\in[L]} s_j}{\hat{n}}\right] = \frac{\sum_{j\in[L]} \mathbb{E}[s_j]}{\hat{n}} = \frac{\sum_{j\in[L]} \mathbb{E}\left[\lfloor \eta \hat{s}_j \rceil\right]}{\hat{n}} = \frac{\eta \sum_{j\in[L]} \hat{s}_j}{\hat{n}} = \frac{\eta \hat{n}}{\hat{n}} = \eta. \tag{2.2}$$

We now create the d-dimensional reference layer. One may think of this reference layer as a latent space representing node properties (such as people's age, education, geographic location, beliefs, etc.) and shaping communities (such as communities in social media) based on proximity. Each node $(v_i, i \in [\hat{n}])$ is given a vector $\mathbf{v}_i$ in $\mathbb{R}^d$ sampled independently and uniformly at random from the ball of radius 1 centered at the origin. Let $R_1 = \{\mathbf{v}_i : i \in [\hat{n}]\}$. We sequentially build communities $\hat{C}_1, \hat{C}_2, \ldots, \hat{C}_L$ where, by convention, $\hat{C}_0 = \emptyset$. To create $\hat{C}_j$, we first select the vector in R_j that is furthest from the origin, and call it the seed c_j. The community $\hat{C}_j$ is set as the node associated with the seed, along with the nodes of the seed's $\hat{s}_j - 1$ nearest neighbours in R_j. Finally, we set $R_{j+1} = R_j - \{\mathbf{v}_i : v_i \in \hat{C}_j\}$. At the end of the loop, each node belongs to exactly one primary community and the collection $(\hat{C}_j, j \in [L])$ is a partition of the nodes.

Next, we grow each primary community $\hat{C}_j$ of size $\hat{s}_j$ to the full community C_j of size s_j. Each community will grow independently, so we may grow them in any order (or in parallel). Let $\mathbf{x}_j \in \mathbb{R}^d$ be the center of mass (mean) of $\{\mathbf{v}_i : v_i \in \hat{C}_j\}$. The full community C_j is then the union of $\hat{C}_j$, and the $s_j - \hat{s}_j$ nodes not in $\hat{C}_j$ with vectors closest to $\mathbf{x_j}$. In effect, we have expanded $\hat{C}_j$ around it's center of mass. We call the nodes in $C_j - \hat{C}_j$ the *secondary* members of community j, and likewise for node $v \in C_j - \hat{C}_j$ we call C_j a *secondary* community of v.

We now have each $v_1, \ldots v_{\hat{n}}$ assigned to exactly one primary community and possibly some secondary communities. In Figure 2.2 we show an example of the reference layer with $\hat{n} = 150$ and show the construction of C_3, with $\hat{s}_3 = 40$ and $\eta = 1.75$. From a computational perspective, to ensure that finding primary and secondary communities is efficient, we use k-d trees to perform spatial lookups [11].

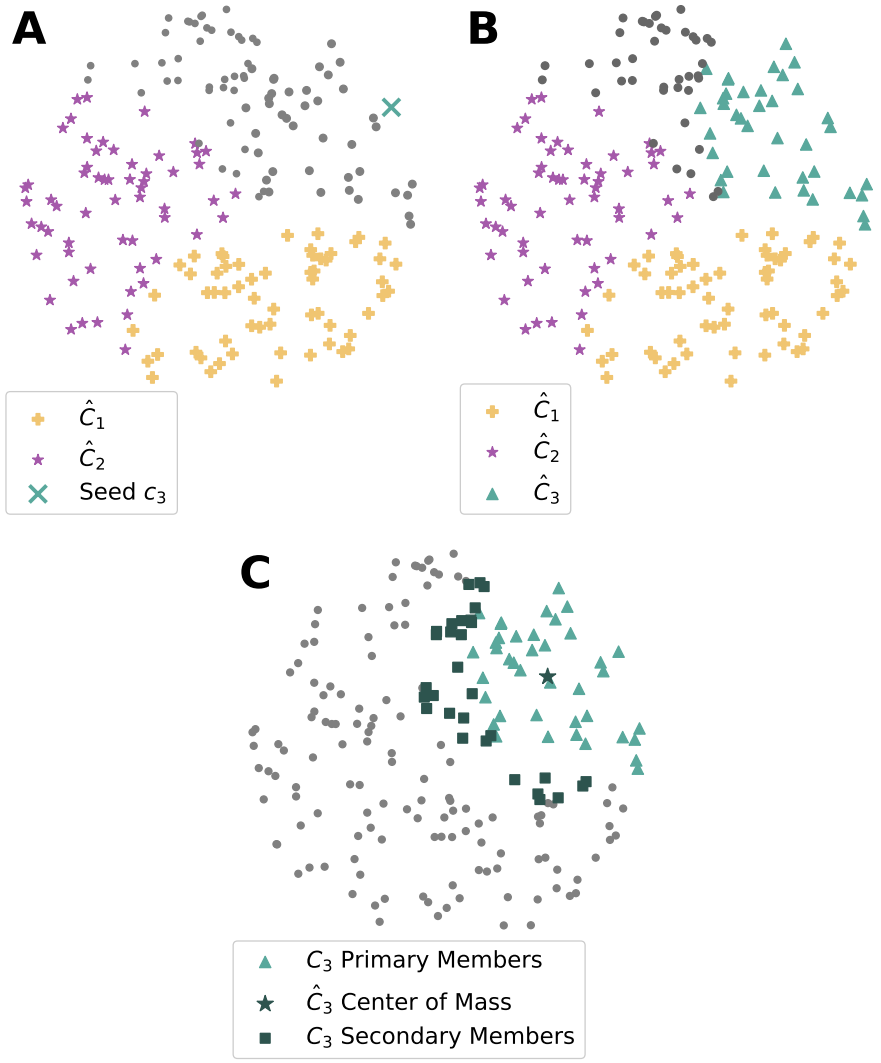


Figure 2.2: Example of building the community C_3 with $\hat{s}_3 = 40$ and $\eta = 1.75$ using a 2-dimensional reference layer. First, in panel A, we select the seed c_3 as the vector furthest from the origin that does not have a primary community. Next (panel B), we build the primary community $\hat{C}_3$ as node of the seed and the nodes of the seed's $\hat{s}_3 - 1$ nearest neighbours that do not yet have a primary community. Finally, in panel C, we expand create C_3 by expanding $\hat{C}_3$ by a factor of η (from 40 to $40 \cdot 1.75 = 70$ nodes) by taking the nearest neighbours to the primary center of mass.

Phase 2: assigning degrees.

By default, we sample the degree sequence $(d_i, i \in [n])$ independently from $\mathcal{P}(\gamma, \delta, \Delta)$. Since the total degree in a simple graph must be even, if $\sum_{i\in[n]} d_i$ is odd, we decrease a maximal d_i by 1. Alternatively, the degree sequence can be given explicitly as an input. Finally, we relabel the sequence so that $d_1 \geq d_2 \geq \ldots, \geq d_n$.

The goal of outliers is to sample their neighbours from the entire graph, ignoring the underlying community structure. However, a problem may occur if ξ is close to zero. In the extreme case, where $\xi = 0$, the edges in the background graph are precisely the edges connecting two outliers, and for a simple background graph to exist all outliers must have degree at most $s_0 - 1$. To handle the more general issue that arises when ξ is small, we estimate the number of nodes with at least one edge in the background graph to upper-bound the allowable degree of an outlier. If we imagine the degrees were assigned at random, we would expect

$$\ell = (\hat{n}/n) \sum_{i\in[n]} \min(1, \xi d_i) \tag{2.3}$$

non-outlier nodes with positive background degree (a non-outlier with degree d will have background degree $\lfloor \xi d \rceil$, see Phase 3). Moreover, since the s_0 outliers will assign their entire degree to the background graph, in total we expect $\ell + s_0$ nodes with positive background degree. Therefore, like in the ABCD+o model [38], we insist that a degree d cannot be assigned to an outlier unless

$$d \leq \ell + s_0 - 1. \tag{2.4}$$

In practice, the number of nodes n is large, the number of outliers s_0 is relatively small, and the level of noise ξ is not zero, so there are plenty of nodes with a non-zero degree in the background graph, and so there is no restriction needed for outliers. A subset of

degrees satisfying (2.4) is selected uniformly at random and assigned to the outliers.

Let $(\hat{d}_i, i \in \hat{n})$ be the subsequence of degrees not assigned to outliers. Similar to previous paragraph, to build simple community graphs we must avoid large degrees being assigned to nodes in small communities. Let $\eta_i = |\{j : v_i \in C_j\}|$ be the number of communities containing v_i. When creating edges in Phase 3, node v_i with degree d will have at most degree $\left\lceil \lceil (1-\xi)d \rceil / \eta_i \right\rceil$ assigned to each of its communities (we split the community degree as evenly as possible between communities). So, if v_i is assigned degree d, then for each community C_j containing v_i, $|C_j|$ must be at least $\left\lceil \lceil (1-\xi)d \rceil / \eta_i \right\rceil + 1$ to accommodate the community neighbours of v_i. This is a lower bound, as additional neighbours of v_i coming from the background graph might end up in C_j as well. To account for the extra background neighbours, the ABCD+o model [35] uses a small correction guided an internal parameter ϕ. Extending the derivation of ϕ to overlapping communities would require analysis of each overlap, which would be computationally expensive. Instead, we keep the same equation as ABCD+o, and let

$$\phi = 1 - \sum_{j \in [L]} \left(\frac{\hat{s}_j}{\hat{n}} \right)^2 \frac{\hat{n}\xi}{\hat{n}\xi + s_0}.$$

Typically, ϕ is very close to 1 and so this correction tends to be negligible in both theory and in practice, but for consistency we keep ϕ in the ABCD+o^2 model.

We can now describe the pairing process of unassigned degrees $(\hat{d}_i, i \in [\hat{n}])$ to non-outlier nodes $(v_k, k \in [\hat{n}])$. To guide the correlation between degree and number of communities, we introduce an internal parameter $\alpha \in \mathbb{R}$ that we discuss in the following paragraph. Recall that $(\hat{d}_i, i \in [\hat{n}])$ is monotone decreasing. We assign degrees sequentially, starting with $\hat{d}_1$. To assign $\hat{d}_i$, let U_i be the collection of nodes without an assigned degree. We assign $\hat{d}_i$ to node v, chosen from the subset of U_i satisfying

$$\hat{d}_i \leq \frac{\eta_k}{1-\xi\phi} \cdot \min\left\{ |C_j| - 1 : v_k \in C_j \right\}, \tag{2.5}$$

with probability proportional to η_k^α. If no nodes in U_i satisfy (2.5), we sample v from the subset of U_i for which the right side of (2.5) is maximal.

After a pairing, we compute the obtained correlation ρ_α and determine if α should be larger ($\rho_\alpha < \rho$) or smaller ($\rho_\alpha > \rho$). We find an appropriate value for α with a binary search, starting with $\alpha_{\min} = -60$ and $\alpha_{\max} = 60$ and stopping once the desired precision is achieved, or after the obtained correlation level stops improving. In some cases it is impossible to reach the correlation level required by the user, in which case the closest possible correlation is produced. The relationship (beyond monotone) between ρ and α is unclear, but we feel it is easier for a user to reason about a desired ρ.

Phase 3: generating graphs.

First, we split each degree into the community degrees and the background degree. For each non-outlier node v_i with degree d_i, define random variables $Y_i = \lfloor (1-\xi)d_i \rceil$ and $Z_i = d_i - Y_i$ (note that $[Y_i] = (1-\xi)d_i$ and $\mathbb{E}[Z_i] = \xi d_i$) and split the degree d_i into Y_i community degree and Z_i background degree. The community degree is further split as evenly as possible into the η_i communities of v_i. Specifically, when splitting the community degree of v_i, $Y_i - \eta_i \lfloor Y_i/\eta_i \rfloor$ communities (chosen randomly) each receive degree $\lceil Y_i/\eta_i \rceil$ and the remaining communities each receive degree $\lfloor Y_i/\eta_i \rfloor$. Recall that by (2.5), we have already ensured that each community containing v_i has at least $\lceil Y_i/\eta_i \rceil + 1$ nodes. For the outliers, the entire degree is assigned to the background.

For each $j \in [L]$, we independently construct the *community graph* $G_{n,j}$ as follows. From the previous paragraph, each $v_i \in C_j$ has some degree allocated to community C_j, and form the community graph $G_{n,j}$ by passing these degrees to the configuration model (Section 2.1.3). In the event that the sum of degrees in a community is odd, we pick a node with maximum degree in this community, decrease the community degree by one, and add one to the background degree. Finally, we construct the *background graph* $G_{n,0}$ as per the configuration model on node set $[n]$ and degree sequence $(Z_i, i \in [n])$.

Phase 4: rewiring self-loops and multi-edges.

Note that, although we are calling $G_{n,0}, G_{n,1}, \ldots, G_{n,L}$ *graphs*, they are in fact *multi-graphs*. To ensure that G_n is simple, we perform a series of *rewirings*. A rewiring takes two edges as input, splits them into four half-edges, and creates two new edges distinct from the input. We first rewire each community graph ($G_{n,j}, j \in [L]$), and the background graph $G_{n,0}$, independently as follows.

1. For each edge $e \in E(G_{n,j})$ that is a loop, we add e to a *recycle* list that is assigned to $G_{n,j}$. Similarly, if $e \in E(G_{n,j})$ contributes to a multi-edge, we add all but one copy of this edge to the *recycle* list.

2. We shuffle the *recycle* list and, for each edge e in the list, we choose another edge e' uniformly from $E(G_{n,j})\backslash\{e\}$ (not necessarily in the list) and attempt to rewire these two edges. If the rewiring does not lead to any further self-loops or multi-edges, we save the result and remove e (and e' if present) from the recycle list.

3. If the recycle list is smaller than before the previous step, we repeat step 2. Otherwise, we give up and move all the "bad" edges from the community graph to a *global recycle* list.

As a result, ignoring edges in the *global recycle* list, all community graphs (including the background graph) are simple. However, as is the case in the original ABCD model, an edge in the background graph can form a multi-edge with an edge in a community graph. Another problem, specific to ABCD+o^2, is that an edge from one community can form a multi-edge with an edge from a different but overlapping community. Both types of problematic edges are added to the *global recycle* list. We merge all community graphs with the background graph, and rewire the *global recycle* list following the same process as before (steps 2 and 3 above with $G_{n,j}$ replaced by the full graph). Since the graph is typically sparse, this final rewiring is usually fast. However, in rare cases it is

possible that the process does not terminate. We set a limit to attempt rewiring at most 100 times the size of the global recycle list. If, after this many iterations, we cannot find any new admissible edge (thus reducing the size of global recycle list) we throw an error and indicate to the user that the algorithm failed to terminate with success for the given input parameters (indicating that the likely cause is that the required input degree sequence was not graphic, and should be reconsidered by the user).

2.2 Comparing the Model to Real Data

In this section, we present experiments highlighting the properties of the ABCD+o^2 model. We are mainly interested in qualitatively comparing the model's properties with properties of real-world networks. Thus, for the model parameters, we use values derived from several networks that have known overlapping communities. The networks were analyzed in [65] and are available on the Stanford Network Analysis Project (SNAP) website [46].

DBLP: a network where nodes are authors and an edge exists between two authors if they published a paper together. The ground-truth communities are defined by the publication venues of the journals in which the authors published.

Amazon: a network where nodes are products and an edge exists between two products if they are frequently co-purchased. The ground-truth communities are defined by product categories.

YouTube: a network where nodes are YouTube channels and an edge exists between two channels if they are friends on the platform. The ground-truth communities are user-generated social groups. In the full network over 95% of nodes are outliers and the level of noise ξ is 0.96. We consider the subgraph induced by the non-outlier nodes to avoid such extreme values.

Parameter	Description	DBLP	Amazon	YouTube
n	Number of nodes	317,080	334,863	52,675
s_0	Number of outliers	56,082	17,669	0
η	Average number of communities a non-outlier node is part of	2.76	7.16	2.45
ρ	Pearson correlation between the degree of nodes and the number of communities they belong to	0.76	0.22	0.37
γ	Exponent of power-law degree distribution	2.30	3.04	1.87
δ	Min degree	5	5	5
Δ	Max degree	343	549	1,928
β	Exponent of power-law community size distribution	1.88	2.03	2.13
s	Min community size	10	10	10
S	Max community size	7,556	53,551	3,001
ξ	Level of noise	0.11	0.11	0.59

Table 2.2: The parameters used for generating ABCD+o^2 graphs based on the DBLP, Amazon, and YouTube datasets.

For each network, all parameters other than δ and s are measured directly from the data. We set $\delta = 5$ to avoid cases where nodes have no neighbours in their own community, and insist on $s \geq 10$. The power-law exponents γ and β are fit using maximum-likelihood estimation described in [18] and implemented in the Python package **powerlaw** [2]. In this fitting process, we set the minimum degree as $\delta = 5$ and the minimum community size as $s = 10$ for consistency. The level of noise ξ is computed directly from the graphs as the fraction $|E_b|/|E|$ where E_b is the set of edges where the incident nodes share no communities. The parameters for each network are shown in Table 2.2.

Additionally, most of the experiments are affected by the dimension d of the hidden reference layer. In such cases, we compare dimensions 2, 8, and 64.

2.2.1 Effects of the Reference Layer

The new and novel feature of the ABCD+o^2 model is the hidden reference layer used to generate overlapping communities. Here, we analyze the effects of this reference layer on the community overlap sizes. For comparison, we consider the CKB model [17] following a description from [57]. The CKB model is an application of the random bipartite community affiliation graph proposed for overlapping LFR [40] that uses a power-law distribution for the number of communities per node. Let us briefly summarize this graph construction process. The community affiliation graph is a bipartite graph where one part consists of nodes and the other consists of communities. As input, the CKB model requires the number of nodes n, a truncated power-law distribution $\mathcal{P}(\Omega, x_{\min}, x_{\max})$ for the number of communities per node, and another truncated power-law distribution $\mathcal{P}(\beta, s, S)$ for the community sizes. The number of communities is set to

$$\left\lfloor \frac{n \cdot \mathbb{E}[\mathcal{P}(\Omega, x_{\min}, x_{\max})]}{\mathbb{E}[\mathcal{P}(\beta, s, S)]} \right\rfloor .$$

First, each node in the bipartite graph is assigned a number of half-edges sampled from $\mathcal{P}(\Omega, x_{\min}, x_{\max})$, and likewise each community is assigned a number of half-edges sampled from $\mathcal{P}(\beta, s, S)$. A small adjustment is made to ensure that the number of half-edges in both parts is equal. Lastly, the nodes are matched with the communities based on the bipartite configuration model. Thus, each community is populated with nodes, and each node is assigned to some number of communities.

Community sizes.

In the first experiment (see Figure 2.3), we compare the distributions of community sizes. Of course, since both ABCD+o^2 and CKN sample community sizes directly from the specified distribution, it is unsurprising that the empirical distributions are near-perfect fits when accounting for the imposed minimum community size of 10. Thus, this

first experiment merely acts as a sanity check ensuring that the ABCD+o^2 and CKN models generate community sizes correctly. Moreover, this experiment verifies that the distributions coming from the real networks are indeed power-law.

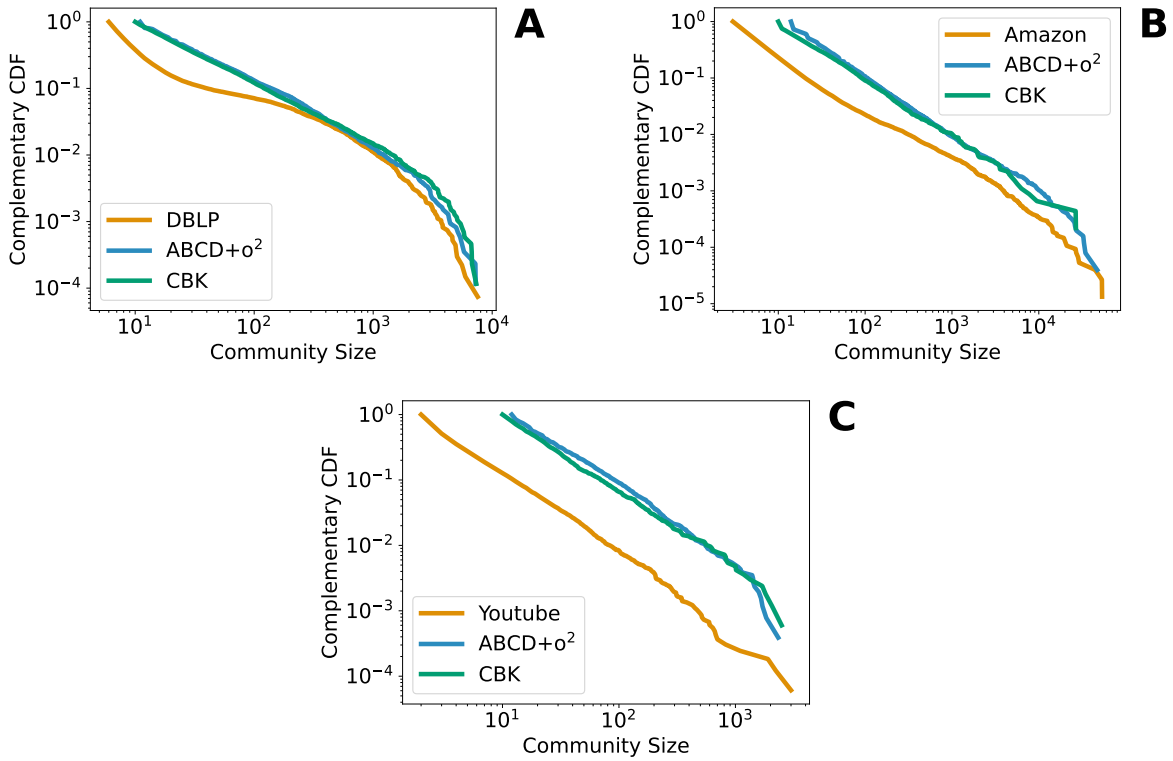


Figure 2.3: Distribution of community sizes for real-world networks and their synthetic counterparts: DBLP in panel A, Amazon in panel B, and Youtube in panel C. The y-axis shows the empirical complementary cumulative distribution function, computed as $y = 1 - |\{C_i : |C_i| < x\}| \,/\, |\{C_i\}|$.

Number of communities per node.

Results are presented in Figure 2.4. In contrast with the previous experiment, there is less agreement between the two models and the real distributions. In the case of the YouTube and DBLP networks, the power-law distribution generated by the CKN model is a good

fit for the empirical distribution, whereas the Amazon distribution does not appear to be power-law. In contrast, the ABCD+o^2 model is able to produce a distribution similar to each of the three networks with a properly tuned dimension parameter d; for the Amazon graph a low dimensional reference layer captures the behaviour well, and for the DBLP and YouTube graphs a high dimensional reference layer captures the behaviour well. The flexibility and realistic distributions are even more encouraging when we recall that the distribution is not given to the ABCD+o^{2}model, and is instead a natural feature of the hidden reference layer.

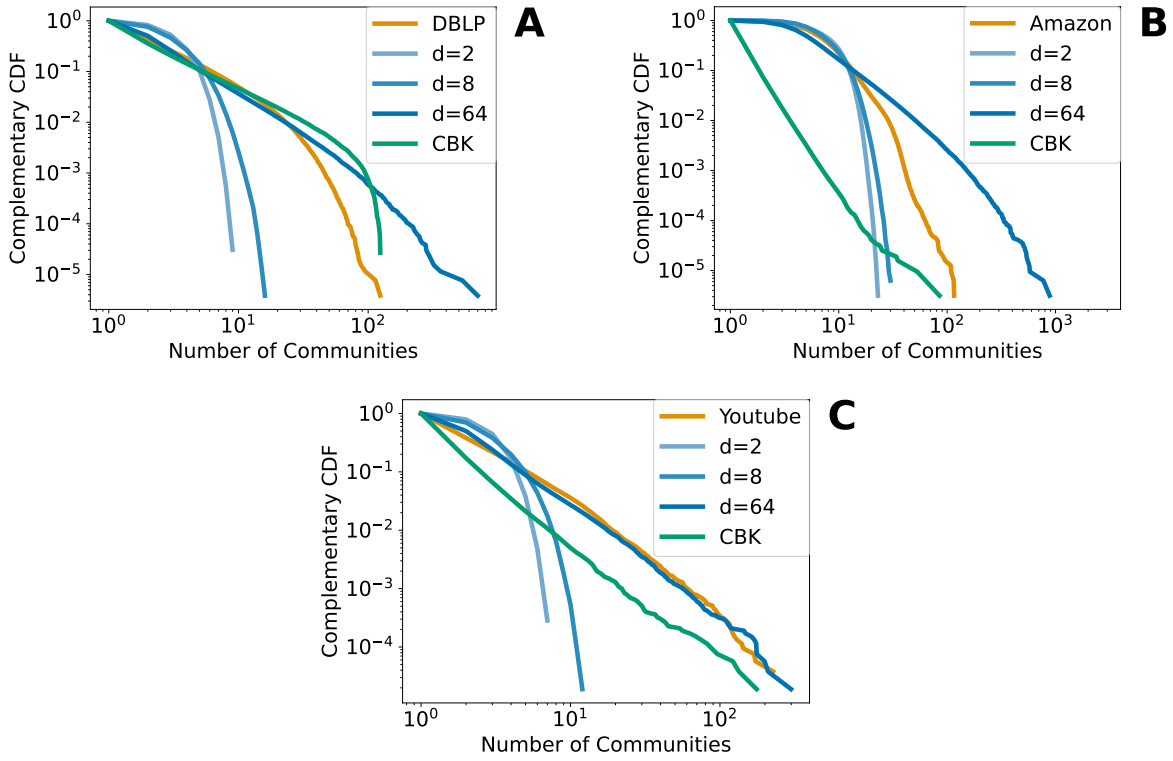


Figure 2.4: Distribution of the number of communities per node in each real-world network and their synthetic counterparts: DBLP in panel A, Amazon in panel B, and Youtube in panel C. The y-axis shows the empirical complementary cumulative distribution function, computed as $y = 1 - |\{v_i : |\{C_j : v_i \in C_j\} < x\}| \,/\, |V|$.

Community intersection sizes.

Results are presented in Figure 2.5. We examine the size of the overlaps produced by each model. Neither the ABCD+o^2 model nor the CKN model specifies a particular distribution, but we can see the ABCD+o^2 model, with a well-tuned dimension parameter, is able to create a similar distribution to the real graph. In contrast, the CKN model does not fit the Amazon and YouTube graphs well. Specifically, the CKN model produces too many small intersections and, moreover, the largest intersection is several orders of magnitude too small. We consider this experiment strong evidence for using a reference layer as the default option for generating overlapping communities.

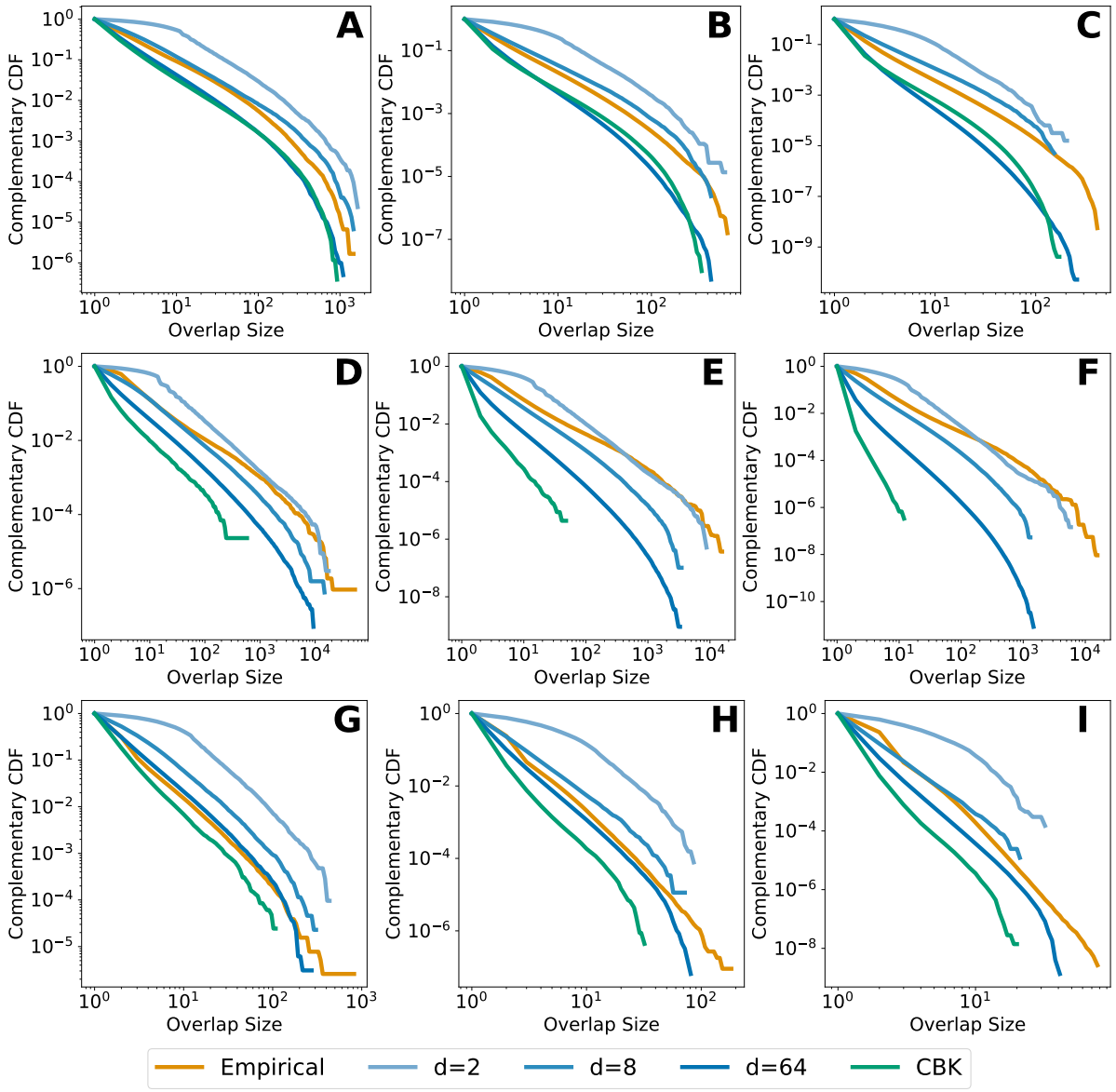


Figure 2.5: Distribution of the size of overlaps in real graphs and their synthetic counterparts. Each row corresponds to a real graph, panels A,B,C for DBLP, panels D,E,F for Amazon, and panels G,H,I for Youtube. The left (panels A,D,G), center (panels B,E,H) and right (panels C,F,I) columns correspond to overlaps of two, three, and four communities respectively. The y-axis shows the empirical complementary cumulative distribution function of non-empty overlaps. For example, the center column of figures corresponds to three overlapping communities and is computed as $y = 1 - |\{\{C_i, C_j, C_k\} : |C_i \cap C_j \cap C_k| < x\}| \,/\, |\{\{C_i, C_j, C_k\} : |C_i \cap C_j \cap C_k| \neq \emptyset|$.

Graph	Empirical	$d = 2$	$d = 8$	$d = 64$
DBLP	0.76	0.42	0.56	0.68
Amazon	0.22	0.20	0.19	0.20
YouTube	0.37	0.37	0.36	0.38

Table 2.3: Measured ρ in both the real networks and the corresponding ABCD+o^2 graphs. We generated 10 ABCD+o^2 graphs for each dimension d of the reference layer, and found that across all graphs and dimensions, the standard error, to two decimal places, was 0.

2.2.2 Additional properties of the model.

We now turn to properties of the ABCD+o^2 model beyond those governed by the hidden reference layer.

Degree vs. number of communities.

The ABCD+o^2 model attempts to produce a correlation ρ between the degree of a node and the number of communities it belongs to. In Table 2.3 we report the desired correlation ρ for each of the networks, as well as the achieved correlation for the ABCD+o^2 graphs. Note that the extreme positive correlation found in the DBLP network cannot be matched by the model, although it can still achieve a strong positive correlation. The maximum achievable correlation increases with the dimension as there are more nodes belonging to a large number of communities (see Figure 2.4).

Density of intersections

One consequence of forcing nodes of large degree into community intersections is that those intersections will be denser than the individual communities. We show this phenomenon in Figure 2.6 by comparing the community and intersection densities for various ρ while fixing the other parameters. Note that, given communities C_j and C_k with $s_j \gg s_k$, the density of C_k is almost always larger than that of C_j since both community graphs are sparse (at most a $1 - \xi$ fraction of each node's degree is delegated to

one of its communities). Figures for the other graphs and dimensions can be found in Appendix A.1.

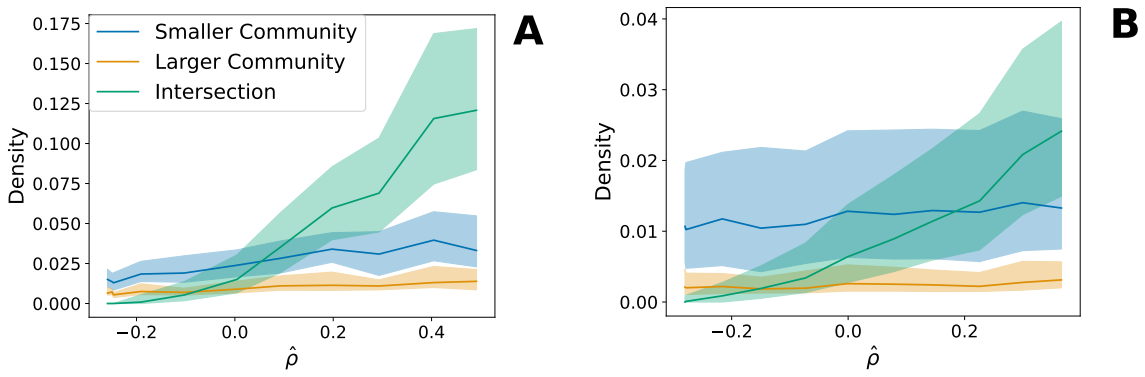


Figure 2.6: Community overlap densities compared to individual community densities for various values of ρ in Youtube-like graphs (panel A) and DBLP-like graphs (panel B) with $d = 8$. For 9 ABCD+o^2 graphs with ρ evenly spaced in $[-0.5, 0.5]$, we compute the empirical value $\hat{\rho}$ and the distribution of density for overlapping communities. We plot $\hat{\rho}$ against the distribution of the density for the intersection and each community individually. The line corresponds to the median, and the shaded region to the 25th to 75th quantile. For computational reasons, we only consider overlaps of size at least 25, and to ensure we are not simply measuring the density of the smaller community, the size of the overlap must be at most half the size of the smaller community.

Having community intersections that are denser than the respective communities is a desirable property and one of our main motivators for introducing the parameter ρ to the ABCD+o^2 model. In [66] it was found that, in real networks, community intersections can be significantly denser than the individual communities. In Figure 2.7 we confirm that the DBLP, Amazon, and YouTube networks all exhibit this phenomenon. Moreover, we compare these densities with those of the corresponding ABCD+o^2 graphs. Aside from confirming that the model can produce behaviour similar to the real networks, we also

find that the dimension of the reference layer affects the density of both the communities and the intersections. Moreover, we see that a low-dimensional reference layer yields a better approximation of the real data in the case of Amazon, whereas a high-dimensional reference layer yields a better approximation in the cases of DBLP and YouTube. In the case of DBLP, the difference in distribution between dimensions is partially attributable to the difference in the empirical ρ (recall with Table 2.3 that the low dimension graphs could not achieve a large correlation. Note that the most similar dimension for each real graph is consistent with the experiment presented in Figure 2.4.

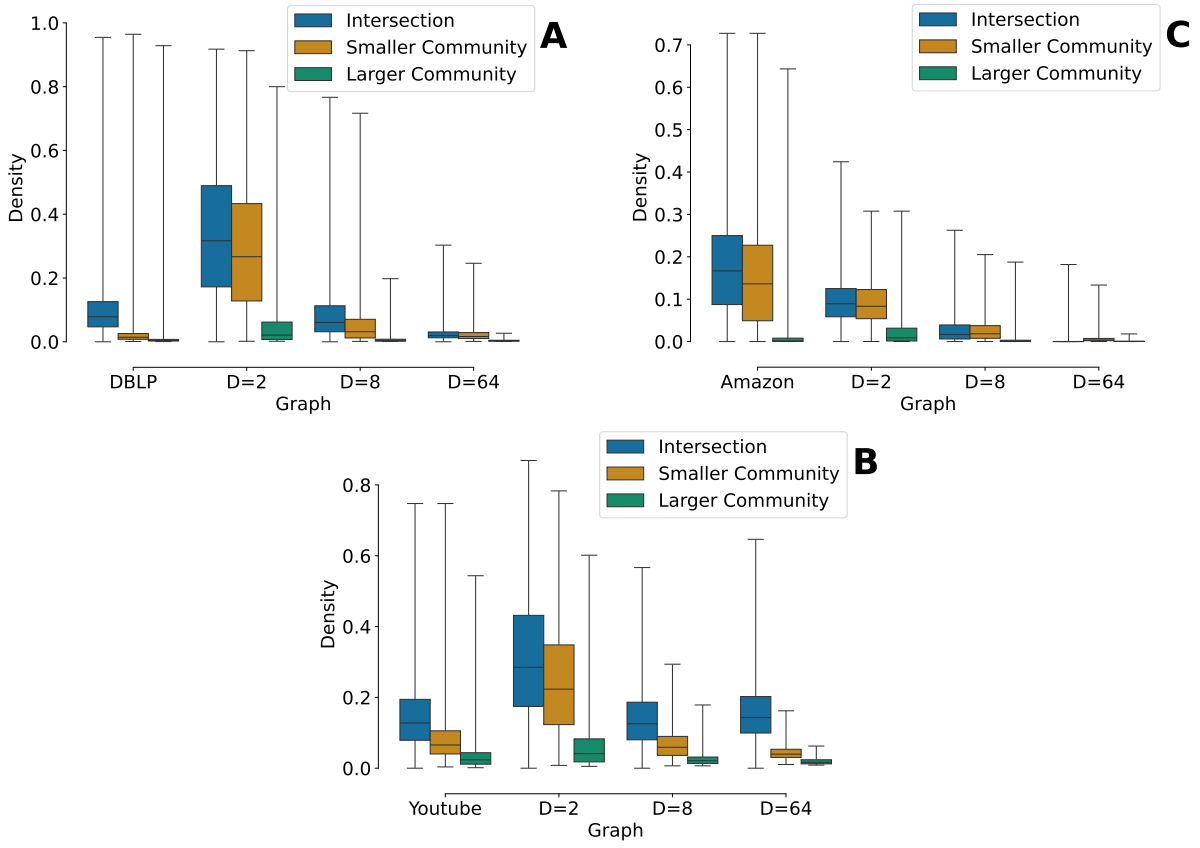


Figure 2.7: Distribution of the density for pairs of communities with non-zero overlap in real graphs and their synthetic counterparts; DBLP in panel A, Amazon in panel B, and Youtube in panel C. The shaded region corresponds to the 25th to 75th quartiles, and the whiskers correspond to the maximum and minimum values. We restrict to overlaps of size at least 10 due to the discrete and unpredictable nature of small communities.

Internal edge fraction

Finally, we analyze how strongly nodes are connected to their communities. There are many measures for community association strength, some of which we recently analyzed in [6]. For the sake of simplicity, the measure we will use here is the internal edge fraction

IEF(v, C), defined as

$$\text{IEF}(v, C) = \frac{|\{\{u, v\} \in E : u \in C\}|}{\deg(v)}\,.$$

For a graph G, a collection of communities $\{C_i, i \in [k]\}$, and a node v, let $C_1^{(v)}, \ldots, C_k^{(v)}$ be an ordering of the communities such that

$$\text{IEF}(v, C_1^{(v)}) \geq \text{IEF}(v, C_2^{(v)}) \cdots \geq \text{IEF}(v, C_k^{(v)})\,.$$

Then, if v is a member of ℓ communities, we would expect a jump in value between IEF$(v, C_\ell^{(v)})$ and IEF$(v, C_{\ell+1}^{(v)})$. In Figure 2.8 we show the range of the top five internal edge fractions, sorted in descending order, on DBLP (left) and a generated ABCD+o^2 graph (right), binned by the number of communities the node v is in. As predicted, a node in ℓ communities tends to have ℓ relatively large internal edge fractions. Furthermore, despite the ABCD+o^2 model splitting the community degree of a node v evenly among its ℓ communities, it is possible for v to have a higher internal edge fraction into community C if either some of v's edges lies in an intersection containing C or some of v's background edges land in C. Thus, we see a pattern in ABCD+o^2 graphs where there are ℓ large but unequal internal edge fraction values. This feature is also present in the DBLP graph, the other real graphs, and the other ABCD+o^2 graphs (see Appendix A.2 for a complete collection of figures).

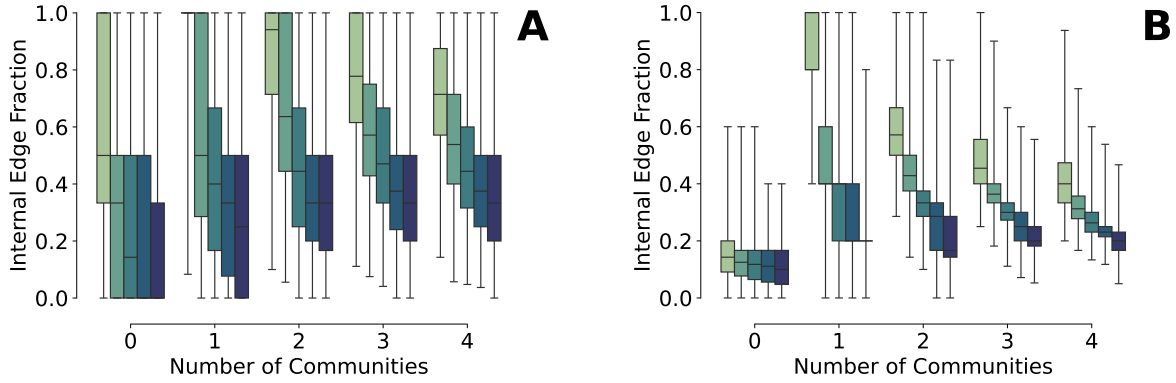


Figure 2.8: Five largest internal edge fractions, sorted in descending order, grouped by the number of communities. Panel A corresponds to the DBLP graphs, and panel B to the synthetic counterpart with reference layer dimension $d = 8$. A pattern is consistent between DBLP and ABCD+o^2, namely, that for nodes belonging to ℓ communities there are ℓ large, but not equal, internal edge fraction values.

Chapter 3

A Pragmatic Method for Comparing Clusterings with Overlaps and Outliers

The second necessary component for extrinsic validity benchmarks is a method to compare clusterings [29]. In this chapter, we use a set-matching framework [56] and the F^* score [30] to define a method for comparing clusterings that could have overlaps or outliers. The method is simple, computationally efficient, and exhibits several intuitive properties one would expect in a comparison measure.

This method works for comparing clusterings over any type of data, so we consider a generic set of objects $X = \{x_1, x_2, \ldots, x_n\}$ to be clustered, and define a *clustering* as set of non-empty subsets of X. We call each element of a clustering a *cluster*. Note that with this definition, each cluster in a clustering is distinct, and the empty set cannot be a cluster. However, we allow the clustering itself to be the empty set.

3.1 Existing Options

Comparing partitions is a well-studied problem, and there are a wide variety of measures that are used [29], which can be broadly classified as either pair-counting, information theoretic, or set-matching [56]. In this section, we review some extensions to overlapping clusterings and discuss the so-called *problem of matching*.

3.1.1 Omega Index

The *Omega* index [19] extends the pair-counting Adjusted Rand Index (ARI) [33] by extending the contingency table from agree versus disagree to counting the number of clusters in common. The adjusted index Ω consists of an unadjusted index ω that is "corrected" for agreement produced by random clustering via

$$\Omega(\mathcal{C}^{(1)}, \mathcal{C}^{(2)}) = \frac{\omega(\mathcal{C}^{(1)}, \mathcal{C}^{(2)}) - \mathbb{E}[\omega(\mathcal{C}^{(1)}, \mathcal{C}^{(2)})]}{\max\{\omega\} - \mathbb{E}[\omega(\mathcal{C}^{(1)}, \mathcal{C}^{(2)})]}.$$

The unadjusted Omega Index is the proportion of pairs of objects that are in the same number of clusters in $\mathcal{C}^{(1)}$ as in $\mathcal{C}^{(2)}$, or, formally,

$$\omega(\mathcal{C}^{(1)}, \mathcal{C}^{(2)}) = \sum_{x_i, x_j \in \binom{X}{2}} \chi\left\{\left|\{C_1 \in \mathcal{C}^{(1)} : x_i \in C_1\}\right| = \left|\{C_2 \in \mathcal{C}^{(2)} : x_j \in C_2\}\right|\right\}. \tag{3.1}$$

The adjustment follows a formula copied from the ARI, and computes the expected value of the index as

$$\mathbb{E}[\omega(\mathcal{C}^{(1)}, \mathcal{C}^{(2)})] = \sum_{k=0}^{\min\{|\mathcal{C}^{(1)}|, |\mathcal{C}^{(2)}|\}} \frac{A_1^{(k)} \cdot A_2^{(k)}}{|X|^2} \tag{3.2}$$

where $A_1^{(k)}$ and $A_2^{(k)}$ are the number of objects in X that are in k clusters in $\mathcal{C}^{(1)}$ and $\mathcal{C}^{(2)}$ respectively.

This extension seems reasonable, and is equivalent to the ARI for partitions, but is

quite harsh since it does not detect any similarity if a pair of objects do not share the exact same number of clusters in both clusterings. Furthermore, values of ω tends to always be large, even for intuitively dissimilar clusterings, which is the same issue that prompted the adjustment of the Rand Index [50, 33]. The adjustment proposed implicitly assumes some null-model for generating random clusterings, the choice of which has been shown to have an effect for comparing partitions [26], and generally requires careful consideration of the assumptions when extending beyond partitions [22].

3.1.2 Overlapping Normalized Mutual Information

The overlapping Normalized Mutual Information (oNMI) [41] is an extension of the information theoretic Normalized Mutual Information [61] using a set matching framework. To compare two clusters C_i and C_j, define the conditional entropy

$$H(C_i|C_j) = h(a,n) + h(b,n) + h(c,n) + h(d,n) - h(b+d,n) - h(a+c,n) \tag{3.3}$$

where $h(w,n) = -w\log_2\frac{w}{n}$ and $a = |C_i \cap C_j|, b = |C_i \cap C_j^c|, c = |C_i^c \cap C_j|$, and $d = |C_i^c, C_j^c|$. However, this entropy can be minimized by complementary clusters (i.e. $C_i = C_j^c$). Thus, define an "overlap only" conditional entropy

$$H^*(C_i|C_j) = \begin{cases} H(C_i|C_j) & \text{if } h(a,n) + h(d,n) \geq h(b,n) + h(c,n), \\ h(c+d,n) + h(a+b,n) & \text{otherwise.} \end{cases} \tag{3.4}$$

The conditional entropy of a clustering is defined as

$$H(\mathcal{C}^{(1)}|\mathcal{C}^{(2)}) = \sum_{C_i \in \mathcal{C}^{(1)}} \min_{C_j \in \mathcal{C}^{(2)}} H^*(C_i|C_j), \tag{3.5}$$

and the unconditional entropy of a clustering is defined as

$$H(\mathcal{C}^{(1)}) = \sum_{C_i \in \mathcal{C}^{(1)}} \left(h(|C_i|, n) + h(|C_i^c|, n)\right).$$

Finally, the mutual information of two clusterings is

$$I(\mathcal{C}^{(1)} : \mathcal{C}^{(2)}) = \frac{1}{2}(H(\mathcal{C}^{(1)}) - H(\mathcal{C}^{(1)}|\mathcal{C}^{(2)}) + H(\mathcal{C}^{(2)}) - H(\mathcal{C}^{(2)}|\mathcal{C}^{(1)})).$$

The upper bound of the mutual information depends on the particular clusterings, so for clearer interpretation the range is restricted to $[0, 1]$ via a normalization. The original oNMI of Lancichinetti et al. [41] is

$$\text{oNMI}_{\text{LFK}}(\mathcal{C}^{(1)}, \mathcal{C}^{(2)}) = 1 - \frac{1}{2}\left(\frac{H(\mathcal{C}^{(1)}|\mathcal{C}^{(2)})}{H(\mathcal{C}^{(1)})} + \frac{H(\mathcal{C}^{(2)}|\mathcal{C}^{(1)})}{H(\mathcal{C}^{(2)})}\right), \tag{3.6}$$

but McDaid et al. [47] note this normalization leads to unintuitive behaviour and suggest

$$\text{oNMI}_{\text{MGH}}(\mathcal{C}^{(1)}, \mathcal{C}^{(2)}) = \frac{I(\mathcal{C}^{(1)}, \mathcal{C}^{(2)})}{H(\mathcal{C}^{(1)}), H(\mathcal{C}^{(2)})}. \tag{3.7}$$

In fact, there are 5 reasonable normalizations [61] and, similar to the previously mentioned null-models, any choice must be carefully considered. In our experiments, we test both suggested normalizations.

3.1.3 Element Centric Clustering Similarity

By shifting the focus from clusters to objects, the Element-Centric Clustering comparison (ECC) [27] is a new class of method. The method constructs a *cluster-induced element graph* by placing a directed and non-uniformly weighted clique over each cluster (with weights chosen so that the degree of each object is split evenly across its clusters), and comparing the personalized pagerank vectors of each object.

The cluster-induced element graph $CG(\mathcal{C}^{(1)})$ has vertices X and weighted edges

$$w(x_i, x_j) = \sum_{C_i \in \mathcal{C}^{(1)}} \frac{\chi\{x_i, x_j \in C_i\}}{|\{C \in \mathcal{C}^{(1)} : x_i \in C\}| \cdot |\{C \in \mathcal{C}^{(1)} : x_j \in C\}|}, \tag{3.8}$$

where weight 0 implies there is no edge. The personalized page rank vector [52] $p_i^{(1)}$ of vertex x_i is the solution to the diffusion

$$p_i^{(1)} = (1 - \alpha)e_i + \alpha p_i W^{(1)}, \tag{3.9}$$

where e_i is the ith basis vector (all zeros with a 1 in coordinate i), α is restart parameter (fixed as 0.9 as suggested), and $W^{(1)}$ is the weighted adjacency matrix of $CG(\mathcal{C}^{(1)})$. The vector $p_i^{(2)}$ is defined similarly for $\mathcal{C}^{(2)}$. Finally, the similarity is the average normalized L_1 distance between the pagerank vectors from each cluster induced subgraph:

$$ECC(\mathcal{C}^{(1)}, \mathcal{C}^{(2)}) = \frac{1}{|X|} \sum_{x_i \in X} \left(1 - \frac{1}{2\alpha} ||p_i^{(1)} - p_i^{(2)}||_1\right). \tag{3.10}$$

This method allows for interesting object-level agreement analysis, but has several drawbacks for comparing clusterings. Critically, the ECC may assign maximum similarity to non-identical clusterings; for example, a single cluster that contains all objects compared to any uniform Balanced Incomplete Block Design (BIBD) over the objects will achieve maximum similarity. A BIBD is a clustering where each pair of objects appears in the same number of clusters and each cluster has the same size. These symmetries create a cluster induced element graph that is a clique with equal weights, which is identical (up to a constant scaling, which is ignored by the personalized pagerank computation) to the cluster-induced element graph of a single cluster containing all objects. Furthermore, the ECC is computationally difficult for large datasets with overlapping clusters, and the natural extension would treat outliers the same as clusters of size one.

3.1.4 The Problem of Matching

Finally, although not another similarity measure, let us discuss the *problem of matching* [49] that affects set-matching measures [56] like the oNMI and our proposed measure. The problem of matching refers to the indifference of a measure to the part of each cluster that is unmatched. In particular, if the unmatched portion of a cluster is concentrated in one other cluster, then it is somehow less disrupted than if the unmatched portion was spread across many clusters. However, while it may be less disrupted in an edit distance sense (and only if moving multiple objects between clusters costs less than moving each object individually), we disagree that intuition dictates that these disruptions should not be equally dissimilar to the original. In fact, by measuring the ability of each cluster to find a match in the other clustering, a set-matching measure provides interesting cluster level information that could be useful for detecting where the clusterings disagree, possibly complementing the object level information available from the ECC [27].

3.2 The Proposed Measure

Following the set-matching framework, we first require a method for comparing two individual clusters. A cluster similarity function is a map between two subsets of X and a value in $[0, 1]$ that measures some notion of similarity (with 1 denoting maximum similarity). The standard precision (p), recall (r), and F_1 similarity functions between two clusters $C_i, C_j \subseteq X$ are defined as:

$$p(C_i, C_j) = \frac{|C_i \cap C_j|}{|C_i|},$$
$$r(C_i, C_j) = \frac{|C_i \cap C_j|}{|C_j|},$$
$$F_1(C_i, C_j) = \frac{2}{1/p(C_i, C_j) + 1/r(C_i, C_j)}.$$

Precision and recall both have nice interpretations as the probability of being in a cluster conditional on being in other. The F_1 score combines the precision and recall into a single score, and, although popular, has gathered criticism for being pragmatic but ultimately unjustified [16]. In [30], the authors argue that another combination of precision and recall, which they call the F^* score, is more interpretable. Furthermore, they show that F^* is a monotonic transformation of F_1, which means any ranking of similarities using F_1 will be the same when using F^*. The F^* score is equivalent to the well-known Jaccard index for set similarity, and is defined as

$$F^*(C_i, C_j) = \frac{F_1(C_i, C_j)}{2 - F_1(C_i, C_j)} = \frac{|C_i \cap C_j|}{|C_i \cup C_j|}. \tag{3.11}$$

For the rest of the paper, we will use F^*. It could be replaced by any of the above (or other) set similarity measures, but it is easy to work with and will serve as a general comparison measure.

Next, we define methods for measuring how well each cluster in $\mathcal{C}^{(1)}$ can find a matching cluster in $\mathcal{C}^{(2)}$. For a cluster $C_i \in \mathcal{C}^{(1)}$, we find a matching cluster by taking the maximum similarity when comparing to each cluster in $\mathcal{C}^{(2)}$:

$$F^*(C_i, \mathcal{C}^{(2)}) = \max_{C_j \in \mathcal{C}^{(2)}} \left\{ F^*(C_i, C_j) \right\}. \tag{3.12}$$

We define the edge case $F^*(C_i, \emptyset) = 0$ since C_i cannot find a cluster with any similarity.

Then, to summarize the match quality from each cluster, we compute the weighted average similarity.

$$\hat{F}^*_w(\mathcal{C}^{(1)}, \mathcal{C}^{(2)}) = \sum_{C_i \in \mathcal{C}^{(1)}} \frac{|C_i|}{\sum_{C_k \in \mathcal{C}^{(1)}} |C_k|} F^*(C_i, \mathcal{C}^{(2)}). \tag{3.13}$$

This function is a measure of how well each cluster in $\mathcal{C}^{(1)}$ can find a match in $\mathcal{C}^{(2)}$, but not necessarily the reverse. This asymmetric similarity measure could be useful in domains

where the performance of a clustering algorithm should be measured by its ability to find clusters that actually exist, but where we are not concerned with complete coverage; for example, in some instances of exploratory data analysis.

Of course, we can also find the best match for each cluster from $\mathcal{C}^{(2)}$ in $\mathcal{C}^{(1)}$ with $\hat{F}_w^*(\mathcal{C}^{(2)}, \mathcal{C}^{(1)})$. Finally, we use a linear combination with equal weights to combine both directions into a single symmetric similarity score;

$$F_w^*(\mathcal{C}^{(1)}, \mathcal{C}^{(2)}) = 0.5\hat{F}_w^*(\mathcal{C}^{(1)}, \mathcal{C}^{(2)}) + 0.5\hat{F}_w^*(\mathcal{C}^{(2)}, \mathcal{C}^{(1)}). \tag{3.14}$$

The similarity measure F_w^* serves as a simple score to evaluate how similar the clusters of two clusterings are, but crucially does not consider objects that are not part of any cluster. We call an object $x_i \in X$ an *outlier* with respect to a clustering $\mathcal{C}$ if $x \notin \bigcup_{C \in \mathcal{C}} C$. For cases where most of the objects are not outliers, F_w^* might be acceptable, but we can add a term for matching the outliers between clusterings. Let $O^{(1)}$ be the set of outliers with respect to $\mathcal{C}^{(1)}$, and likewise for $O^{(2)}$ with $\mathcal{C}^{(2)}$. The outlier aware versions of $\hat{F}_w^*$ and F_w^* are defined as

$$\hat{F}_{wo}^*(\mathcal{C}^{(1)}, \mathcal{C}^{(2)}) = \frac{|O^{(1)}|}{|X|} F^*(O^{(1)}, O^{(2)}) + \frac{|X \setminus O^{(1)}|}{|X|} \hat{F}_w^*(\mathcal{C}^{(1)}, \mathcal{C}^{(2)}) \tag{3.15}$$

and

$$F_{wo}^*(\mathcal{C}^{(1)}, \mathcal{C}^{(2)}) = 0.5\hat{F}_{wo}^*(\mathcal{C}^{(1)}, \mathcal{C}^{(2)}) + 0.5\hat{F}_{wo}^*(\mathcal{C}^{(2)}, \mathcal{C}^{(1)}). \tag{3.16}$$

A python script for computing F_{wo}^* is available in Appendix B.

3.3 Properties

In this section, we analyze some basic properties of the proposed F_{wo}^* score, and establish that it is reasonable in matching the intuition and general use-case of comparing

clustering algorithms. First, we state a few properties that are absolutely required for an intuitive similarity measure, the proofs of which are elementary and are omitted for brevity.

Proposition 1. *The proposed measure F^*_{wo} has the following properties.*

1. *F^*_{wo} is label invariant.*
2. *$F^*_{wo}(\mathcal{C}^{(1)}, \mathcal{C}^{(2)}) \in [0, 1]$, and $F^*_{wo}(\mathcal{C}^{(1)}, \mathcal{C}^{(2)}) = 1$ if and only if $\mathcal{C}^{(1)} = \mathcal{C}^{(2)}$.*
3. *F^*_{wo} is symmetric.*
4. *Single object clusters are not the same as outliers.*

The first property is the essential difference between clustering and multi-class classification tasks. Implementations generally assign a label to each cluster (often using the natural numbers 1 to $|\mathcal{C}|$), but such a labelling is arbitrary and should not affect the similarity to another clustering. The second property is called *normalization* [61], and it is defined as a bounded range for the measure independent of the particular clusterings under consideration. A normalized measure means that the output is on the same scale regardless of the clusterings, and scores are comparable even if the clusterings have a different number of clusters or objects. The final property is the key differentiation of our method with those presented in Section 3.1, in the sense that the obvious extensions of the existing methods would treat outliers identically to single object clusters.

The complement of some similarity measures (that is, $1 - s$ for a similarity measure s) for partitions are metrics, and it has been argued that a metric structure better matches our intuitions about similarity [49]. However, in practice, many of the most popular similarity measures, such as the ARI [33], NMI [39] (using the arithmetic mean, the default in the popular python machine learning library scikit-learn), and AMI [61], in addition to the Omega [19], oNMI [41], and ECC [27] from Section 3.1, are not the complement of a metric. Thus, even though being the complement of a metric is desirable,

we conclude that this property is not absolutely necessary for a pragmatic similarity measure.

Proposition 2. *The complement of the proposed measure, $1 - F^*_{wo}$, is not a metric.*

Proof. We provide a counterexample for the triangle inequality. Let $M(\mathcal{C}^{(1)}, \mathcal{C}^{(2)}) = 1 - F^*_{wo}(\mathcal{C}^{(1)}, \mathcal{C}^{(2)})$. Let $X = \{1, 2, 3\}$, and consider the following three clusterings:

$$\mathcal{C}^{(1)} = \{\{1, 2, 3\}\}, \quad \mathcal{C}^{(2)} = \{\{1\}, \{1, 2, 3\}\}, \quad \mathcal{C}^{(3)} = \{\{1\}, \{2, 3\}\}$$

We compute

$$\begin{aligned}
M(\mathcal{C}^{(1)}, \mathcal{C}^{(2)}) &= 1 - \left(0.5\,(1) + 0.5\left(\frac{1}{4}\frac{1}{3} + \frac{3}{4}1\right)\right) = \frac{1}{12}, \\
M(\mathcal{C}^{(2)}, \mathcal{C}^{(3)}) &= 1 - \left(0.5\left(\frac{1}{4}1 + \frac{3}{4}\frac{2}{3}\right) + 0.5\left(\frac{1}{3}1 + \frac{2}{3}\frac{2}{3}\right)\right) = \frac{17}{72}, \\
M(\mathcal{C}^{(1)}, \mathcal{C}^{(3)}) &= 1 - \left(0.5\left(\frac{2}{3}\right) + 0.5\left(\frac{1}{3}\frac{1}{3} + \frac{2}{3}\frac{2}{3}\right)\right) = \frac{7}{18},
\end{aligned}$$

and find that

$$M(\mathcal{C}^{(1)}, \mathcal{C}^{(2)}) + M(\mathcal{C}^{(2)}, \mathcal{C}^{(3)}) = \frac{1}{12} + \frac{17}{72} = \frac{23}{72} < \frac{7}{18} = M(\mathcal{C}^{(1)}, \mathcal{C}^{(3)}),$$

and the triangle inequality does not hold. □

Since none of the clusterings used in the proof of Proposition 2 contain outliers, this counterexample shows that $1 - F^*_w$ is also not a metric.

Finally, we examine the effect of a small change to one of the clusterings being compared.

Theorem 3. *Let $\mathcal{C}^{(1)}$ and $\mathcal{C}^{(2)}$ be clusterings of X, and let $\tilde{\mathcal{C}}^{(1)}$ be a clustering created from $\mathcal{C}^{(1)}$ by moving one object, x, into or out of one cluster. Let $B = \sum_{C \in \mathcal{C}^{(2)}} |C|$. If*

$|\mathcal{C}^{(1)}| = |\tilde{\mathcal{C}}^{(1)}|$ *(no clusters were created or destroyed), say* x *was added or removed from* $C_1 \in \mathcal{C}^{(1)}$*, then*

$$\begin{aligned}&\left|F^*_{wo}(\mathcal{C}^{(1)}, \mathcal{C}^{(2)}) - F^*_{wo}(\tilde{\mathcal{C}}^{(1)}, \mathcal{C}^{(2)})\right| \\ &\qquad = \mathcal{O}\left(\frac{1}{|X|}\max\left\{1, \frac{|X \setminus O^{(2)}|\gamma}{B}\right\}\right),\end{aligned}$$

where $\gamma = |\{C \in \mathcal{C}^{(2)} : C \text{ matched with } C_1 \text{ or } x \in C\}|$*. Otherwise,*

$$\left|F^*_{wo}(\mathcal{C}^{(1)}, \mathcal{C}^{(2)}) - F^*_{wo}(\tilde{\mathcal{C}}^{(1)}, \mathcal{C}^{(2)})\right| = \mathcal{O}\left(\frac{1}{|X|}\max\left\{1, \frac{|X \setminus O^{(2)}|\Gamma}{B}\right\}\right),$$

where $\Gamma = |\{C \in \mathcal{C}^{(2)} : x \in C\}|$*.*

With Theorem 3, we confirm that, subject to some mild constraints on the clusterings, moving one element into or out of a cluster has only a small effect on the similarity to a reference clustering. In both cases, the right-hand side can be improved to $\mathcal{O}(\frac{1}{|X|})$ with additional mild constraints on $\mathcal{C}^{(2)}$ that prevent an object from being in too many clusters.

First, we prove a few lemmas about some of the intermediate scores, and we will then combine these results to prove Theorem 3. For rest of the section, let $\mathcal{C}^{(1)}$ and $\mathcal{C}^{(2)}$ be clusterings of X. Let $x \in X \setminus C_1$ for some $C_1 \in \mathcal{C}^{(1)}$, and create a new cluster $\tilde{C} = C_1 \cup \{x\}$. Let $\tilde{\mathcal{C}}^{(1)}$ be the clustering obtained from $\mathcal{C}^{(1)}$ by replacing C_1 with $\tilde{C}$. This corresponds to the first scenario in Theorem 3.

Lemma 4. *If* $C_j \subseteq X$ *then*

$$\left|F^*(C_1, C_j) - F^*(\tilde{C}, C_j)\right| \leq \frac{1}{|C_1|} \text{ and } \left|F^*(C_1, C_j) - F^*(\tilde{C}, C_j)\right| \leq \frac{1}{|C_j|}.$$

Proof. There are two cases to consider. First, if $x \in C_j$, then

$$\left|F^*(C_1, C_j) - F^*(\tilde{C}, C_j)\right| = \left|\frac{|C_1 \cap C_j|}{|C_1 \cup C_j|} - \frac{|\tilde{C} \cap C_j|}{|\tilde{C} \cup C_j|}\right| = \frac{1}{|C_1 \cup C_j|}.$$

And if $x \notin C_j$, then

$$\left|F^*(C_1, C_j) - F^*(\tilde{C}, C_j)\right| = \left|\frac{|C_1 \cap C_j|}{|C_1 \cup C_j|} - \frac{|\tilde{C} \cap C_j|}{|\tilde{C} \cup C_j|}\right| = \frac{|C_1 \cap C_j|}{|C_1 \cup C_j| \cdot |\tilde{C} \cup C_j|} \leq \frac{1}{|\tilde{C} \cup C_j|}.$$

In either case, $|C_1|, |C_j| \leq |C_1 \cup C_j| \leq |\tilde{C} \cup C_j|$ so $1/|\tilde{C} \cup C_j| \leq 1/|C_1 \cup C_j| \leq 1/|C_1|, 1/|C_j|$.

□

Lemma 5. *Let* $A = \sum_{C \in \mathcal{C}^{(1)}} |C|$.

$$\left|\hat{F}_w^*(\mathcal{C}^{(1)}, \mathcal{C}^{(2)}) - \hat{F}_w^*(\tilde{\mathcal{C}}^{(1)}, \mathcal{C}^{(2)})\right| = \mathcal{O}\left(\frac{1}{A}\right).$$

Proof. Recall that

$$\hat{F}_w^*(\mathcal{C}^{(1)}, \mathcal{C}^{(2)}) = \frac{1}{A} \sum_{C \in \mathcal{C}^{(1)}} |C| F^*(C, \mathcal{C}^{(2)}).$$

Using the relationship between $\mathcal{C}^{(1)}$ and $\tilde{\mathcal{C}}^{(1)}$, we get

$$\hat{F}_w^*(\tilde{\mathcal{C}}^{(1)}, \mathcal{C}^{(2)}) = \frac{1}{A+1}\left((|C_1|+1)F^*(C_1, \mathcal{C}^{(2)}) + \sum_{C_i \in \mathcal{C}^{(1)}, i \neq 1} |C_i| F^*(C_i, \mathcal{C}^{(2)})\right).$$

It follows that

$$
\begin{aligned}
&\left|\hat{F}_w^*(\mathcal{C}^{(1)},\mathcal{C}^{(2)}) - \hat{F}_w^*(\tilde{\mathcal{C}}^{(1)},\mathcal{C}^{(2)})\right| \\
\leq &\left|\frac{|C_1|F^*(C_1,\mathcal{C}^{(2)})}{A} - \frac{(|C_1|+1)F^*(\tilde{C}_1,\mathcal{C}^{(2)})}{A+1}\right| \\
&\quad + \left|\frac{1}{A} - \frac{1}{A+1}\right| \cdot \left|\sum_{C_i\in\mathcal{C}^{(1)}, i\neq 1} |C_i|F^*(C_i,\mathcal{C}^{(2)})\right| \\
\leq &\frac{A|C_1|\left|F^*(C_1,\mathcal{C}^{(2)}) - F^*(\tilde{C}_1,\mathcal{C}^{(2)})\right| + \left||C_1|F^*(C_1,\mathcal{C}^{(2)}) - AF^*(\tilde{C}_1,\mathcal{C}^{(2)})\right|}{A\cdot(A+1)} \\
&\quad + \frac{1}{A\cdot(A+1)}\cdot A.
\end{aligned}
$$

Using Lemma 4, we get

$$
\left|\hat{F}_w^*(\mathcal{C}^{(1)},\mathcal{C}^{(2)}) - \hat{F}_w^*(\tilde{\mathcal{C}}^{(1)},\mathcal{C}^{(2)})\right| \;\leq\; \frac{A+A+A}{A\cdot(A+1)} = \mathcal{O}\left(\frac{1}{A}\right),
$$

which finishes the proof. □

Lemma 6. *Let* $B = \sum_{C\in\mathcal{C}^{(2)}} |C|$ *and* $\gamma = \left|\{C \in \mathcal{C}^{(2)} : C \text{ matched with } C_1 \text{ or } x \in C\}\right|$.

$$
\left|\hat{F}_w^*(\mathcal{C}^{(2)},\mathcal{C}^{(1)}) - \hat{F}_w^*(\mathcal{C}^{(2)},\tilde{\mathcal{C}}^{(1)})\right| \leq \frac{\gamma}{B}.
$$

Proof. First, note that if $x \notin C$, then $F^*(C,\tilde{C}) < F^*(C,C_1)$. Thus, any cluster not containing x that does not match with C_1 will not match with $\tilde{C}$. So, if $x \notin C$ and C is

not matched with C_1 then $F^*(C, \mathcal{C}^{(1)}) = F^*(C, \tilde{\mathcal{C}}^{(1)})$. Thus, using Lemma 4 we get

$$\begin{aligned}\left|\hat{F}_w^*(\mathcal{C}^{(2)}, \mathcal{C}^{(1)}) - \hat{F}_w^*(\mathcal{C}^{(2)}, \tilde{\mathcal{C}}^{(1)})\right| &= \left|\sum_{C \in \mathcal{C}^{(2)}} \frac{|C|}{B}\left(F^*(C, \mathcal{C}^{(1)}) - F^*(C, \tilde{\mathcal{C}}^{(1)})\right)\right| \\ &\leq \sum_{C \in \mathcal{C}^{(2)}} \frac{|C|}{B}\left|F^*(C, \mathcal{C}^{(1)}) - F^*(C, \tilde{\mathcal{C}}^{(1)})\right| \\ &\leq \frac{\gamma}{B},\end{aligned}$$

and the proof is finished. □

Now, we can prove Theorem 3.

Proof of Theorem 3. Due to symmetry, without loss of generality, we may assume one of the following four cases.

1a. $x \notin O^{(1)}$ and is added to existing cluster C_1 to make clustering $\tilde{\mathcal{C}}^{(1)}$.

1b. $x \in O^{(1)}$ and is added to existing cluster C_1 to make clustering $\tilde{\mathcal{C}}^{(1)}$.

2a. $x \notin O^{(1)}$ and makes a new cluster $\{x\}$ to make clustering $\hat{\mathcal{C}}^{(1)}$.

2b. $x \in O^{(1)}$ and makes a new cluster $\{x\}$ to make clustering $\hat{\mathcal{C}}^{(1)}$.

Recall that $A = \sum_{C \in \mathcal{C}^{(1)}} |C|$ and $B = \sum_{C \in \mathcal{C}^{(2)}} |C|$.

Case 1a:

$$\begin{aligned}&\left|F_{wo}^*(\mathcal{C}^{(1)}, \mathcal{C}^{(2)}) - F_{wo}^*(\tilde{\mathcal{C}}^{(1)}, \mathcal{C}^{(2)})\right| \\ =&\left|0.5\left(\frac{|X \setminus O^{(1)}|}{|X|}\right)\hat{F}_w^*(\mathcal{C}^{(1)}, \mathcal{C}^{(2)}) + 0.5\left(\frac{|X \setminus O^{(2)}|}{|X|}\right)\hat{F}_w^*(\mathcal{C}^{(2)}, \mathcal{C}^{(1)})\right. \\ &\qquad\left. -0.5\left(\frac{|X \setminus O^{(1)}|}{|X|}\right)\hat{F}_w^*(\tilde{\mathcal{C}}^{(1)}, \mathcal{C}^{(2)}) - 0.5\left(\frac{|X \setminus O^{(2)}|}{|X|}\right)\hat{F}_w^*(\mathcal{C}^{(2)}, \tilde{\mathcal{C}}^{(1)})\right| \\ \leq&\frac{|X \setminus O^{(1)}|}{|X|}\frac{1}{A} + \frac{|X \setminus O^{(2)}|}{|X|}\frac{\gamma}{B},\end{aligned}$$

by Lemmas 5 and 6. And, since $|X \setminus O^{(1)}| \leq A$, we conclude that

$$\left|F^*_{wo}(\mathcal{C}^{(1)}, \mathcal{C}^{(2)}) - F^*_{wo}(\tilde{\mathcal{C}}^{(1)}, \mathcal{C}^{(2)})\right| \leq \mathcal{O}\left(\frac{1}{|X|}\max\left\{1, \frac{|X \setminus O^{(2)}|\gamma}{B}\right\}\right).$$

Case 1b: First, we consider each direction of $\hat{F}^*_{wo}$ individually.

$$\begin{aligned}
&\left|\hat{F}^*_{wo}(\mathcal{C}^{(1)}, \mathcal{C}^{(2)}) - \hat{F}^*_{wo}(\tilde{\mathcal{C}}^{(1)}, \mathcal{C}^{(2)})\right| \\
=&\left|\frac{|O^{(1)}|}{|X|}F^*(O^{(1)}, O^{(2)}) + \frac{|X \setminus O^{(1)}|}{|X|}\hat{F}^*_w(\mathcal{C}^{(1)}, \mathcal{C}^{(2)})\right. \\
&\qquad\left. -\frac{|\tilde{O}^{(1)}|}{|X|}F^*(\tilde{O}^{(1)}, O^{(2)}) - \frac{|X \setminus \tilde{O}^{(1)}|}{|X|}\hat{F}^*_w(\tilde{\mathcal{C}}^{(1)}, \mathcal{C}^{(2)})\right| \\
\leq&\frac{1}{|X|}F^*(O^{(1)}, O^{(2)}) + \frac{|O^{(1)}| - 1}{|X|}\left|F^*(O^{(1)}, O^{(2)}) - F^*(\tilde{O}^{(1)}, O^{(2)})\right| \\
&\qquad + \frac{1}{|X|}\hat{F}^*_w(\tilde{\mathcal{C}}^{(1)}, \mathcal{C}^{(2)}) + \frac{|X \setminus O^{(1)}|}{|X|}\left|\hat{F}^*_w(\mathcal{C}^{(1)}, \mathcal{C}^{(2)}) - \hat{F}^*_w(\tilde{\mathcal{C}}^{(1)}, \mathcal{C}^{(2)})\right| \\
=&\mathcal{O}\left(\frac{1}{|X|}\right).
\end{aligned}$$

And for the other direction,

$$\begin{aligned}
&\left|\hat{F}^*_{wo}(\mathcal{C}^{(2)}, \mathcal{C}^{(1)}) - \hat{F}^*_{wo}(\mathcal{C}^{(2)}, \tilde{\mathcal{C}}^{(1)})\right| \\
\leq&\frac{|O^{(2)}|}{|X|}\left|F^*(O^{(2)}, O^{(1)}) - F^*(O^{(2)}, \tilde{O}^{(1)})\right| + \frac{|X \setminus O^{(2)}|}{|X|}\left|\hat{F}^*_w(\mathcal{C}^{(2)}, \mathcal{C}^{(1)}) - \hat{F}^*_w(\mathcal{C}^{(2)}, \tilde{\mathcal{C}}^{(1)})\right| \\
=&\mathcal{O}\left(\frac{1}{|X|}\frac{|X \setminus O^{(2)}|\gamma}{B}\right).
\end{aligned}$$

So together,

$$\left|F^*_{wo}(\mathcal{C}^{(1)}, \mathcal{C}^{(2)}) - F^*_{wo}(\tilde{\mathcal{C}}^{(1)}, \mathcal{C}^{(2)})\right| = \mathcal{O}\left(\frac{1}{|X|}\max\left\{1, \frac{|X \setminus O^{(2)}|\gamma}{B}\right\}\right).$$

Preliminaries for Case 2a and 2b: For these cases, we need to slightly alter the proofs of Lemmas 5 and 6 for when x creates a new cluster. First, a similar bound to

Lemma 5:

$$\begin{aligned}
&\left|\hat{F}_w^*(\mathcal{C}^{(1)},\mathcal{C}^{(2)}) - \hat{F}_w^*(\hat{\mathcal{C}}^{(1)},\mathcal{C}^{(2)})\right| \\
=&\left|\sum_{C\in\mathcal{C}^{(1)}} \frac{|C|}{A}F^*(C,\mathcal{C}^{(2)}) - \sum_{C\in\mathcal{C}^{(1)}} \frac{|C|}{A+1}F^*(C,\mathcal{C}^{(2)}) - \frac{1}{A+1}F^*(\{x\},\mathcal{C}^{(2)})\right| \\
\leq&\left(\sum_{C\in\mathcal{C}^{(1)}} \frac{1}{A(A+1)} + \frac{1}{A+1}F^*(\{x\},\mathcal{C}^{(2)})\right) \\
=&\mathcal{O}\left(\frac{1}{A}\right).
\end{aligned}$$

And also a similar bound to Lemma 6. In this case, if $x \notin C$, then $F^*(C,\{x\}) = 0$ so C will not match with $\{x\}$. Let $\Gamma = |\{C \in \mathcal{C}^{(2)} : x \in \mathcal{C}^{(2)}\}|$ Thus,

$$\begin{aligned}
&\left|\hat{F}_w^*(\mathcal{C}^{(2)},\mathcal{C}^{(1)}) - \hat{F}_w^*(\mathcal{C}^{(2)},\hat{\mathcal{C}}^{(1)})\right| \\
\leq&\sum_{C\in\mathcal{C}^{(2)}} \frac{|C|}{B}\left|F^*(C,\mathcal{C}^{(1)}) - F^*(C,\hat{\mathcal{C}}^{(1)})\right| \\
\leq&\frac{\Gamma}{|B|}.
\end{aligned}$$

Case 2a:

$$\begin{aligned}
&\left|F_{wo}^*(\mathcal{C}^{(1)},\mathcal{C}^{(2)}) - F_{wo}^*(\hat{\mathcal{C}}^{(1)},\mathcal{C}^{(2)})\right| \\
\leq&\frac{|X\setminus O^{(1)}|}{|X|}\left|\hat{F}_w^*(\mathcal{C}^{(1)},\mathcal{C}^{(2)}) - \hat{F}_w^*(\hat{\mathcal{C}}^{(1)},\mathcal{C}^{(2)})\right| + \frac{|X\setminus O^{(2)}|}{|X|}\left|\hat{F}_w^*(\mathcal{C}^{(2)},\mathcal{C}^{(1)}) - \hat{F}_w^*(\mathcal{C}^{(2)},\hat{\mathcal{C}}^{(1)})\right| \\
=&\mathcal{O}\left(\frac{1}{|X|}\max\left\{1,\frac{|X\setminus O^{(2)}|\Gamma}{B}\right\}\right).
\end{aligned}$$

Case 2b:

$$
\begin{aligned}
&\left|F^*_{wo}(\mathcal{C}^{(1)},\mathcal{C}^{(2)}) - F^*_{wo}(\hat{\mathcal{C}}^{(1)},\mathcal{C}^{(2)})\right| \\
\leq &\frac{1}{|X|}F^*(O^{(1)},O^{(2)}) + \frac{|O^{(1)}|-1}{|X|}\left|F^*(O^{(1)},O^{(2)}) - F^*(\hat{O}^{(1)},O^{(2)})\right| \\
&\quad + \frac{1}{|X|}\hat{F}^*_w(\hat{\mathcal{C}}^{(1)},\mathcal{C}^{(2)}) + \frac{|X\setminus O^{(1)}|}{|X|}\left|\hat{F}^*_w(\mathcal{C}^{(1)},\mathcal{C}^{(2)}) - \hat{F}^*_w(\hat{\mathcal{C}}^{(1)},\mathcal{C}^{(2)})\right| \\
&\quad + \frac{|O^{(2)}|}{|X|}\left|F^*(O^{(2)},O^{(1)}) - F^*(O^{(2)},\hat{O}^{(1)})\right| + \frac{|X\setminus O^{(2)}|}{|X|}\left|\hat{F}^*_w(\mathcal{C}^{(2)},\mathcal{C}^{(1)}) - \hat{F}^*_w(\mathcal{C}^{(2)},\hat{\mathcal{C}}^{(1)})\right| \\
= &\,\mathcal{O}\left(\frac{1}{|X|}\max\left\{1,\frac{|X\setminus O^{(2)}|\Gamma}{B}\right\}\right).
\end{aligned}
$$

□

3.4 Experiments

3.4.1 Intuitive Behaviour

In this section, we recreate and extend an experiment from Gates et al. [27] to check for common biases in the behaviour of F^*_{wo}. The original experiment consists of three series of clustering comparisons that have an expected intuitive behaviour. None of the scenarios have outliers nor overlapping clusters, but a good measure should still match our intuitions for partitions.

For the first experiment, a clustering is created from 1024 objects with 32 clusters of each of size 32. The second clusterings starts as a copy of the first, but a fraction of its objects have their cluster shuffled. We expect that as the fraction shuffled increases, the similarity should be monotone decreasing to some non-zero value.

In the second scenario, the first clustering is again 1024 objects split into 32 clusters of size 32. The second clustering starts with as a different clustering (randomly selected) with 32 clusters of size 32, and evolves by reassigning a random object to a new cluster

chosen proportional to the cluster sizes. As the process proceeds, we expect the cluster sizes to become more heterogeneous (a few large clusters, many small clusters) and we again expect the similarity to be monotone decreasing to some non-zero value.

Finally, we test for bias in the number of clusters. The first clustering has 1024 objects split into 8 clusters of size 128, and the second clustering is chosen uniformly from partitions of 1024 objects into k equally sized clusters. As k increases, we expect the similarity to be monotone decreasing, and for the similarity to be almost 0 in the extreme case of 1024 clusters of size 1.

As seen in Figure 3.1, the behaviour of F^*_{wo} matches our intuition in each experiment.

Next, we test two more scenarios that involve either outliers or overlapping clusters. To generate these clusterings, we use the method from phase 1 of ABCD+o^2 [7], previously described in Section 2.1.5.

In the fourth scenario, for 25 samples, we generate an underlying geometric layer and initial partition, and then compare various values of η to a reference clustering where $\eta = 3$. Since the second stage of the algorithm is deterministic, we expect perfect similarity when $\eta = 3$ and decreasing similarity as $|\eta - 3|$ increases.

In the fifth scenario, we reverse second stage of the method and shrink each cluster around its center of mass (this can be thought of as setting $\eta < 1$). Since we start with a partition, there will be outliers, but no overlapping clusters. We compare to a reference clustering corresponding to $\eta = 0.5$. This time we test two out of five measures, as only F^*_{wo} is explicitly defined for outliers, and ECC can accommodate outliers by treating them as isolated vertices in the cluster-induced element graph.

Once again, as seen in Figure 3.2, F^*_{wo} behaves as intuition would suggest. The ECC also follows the intuition in both scenarios, although the similarity for highly overlapping clusterings (large η) is rather high.

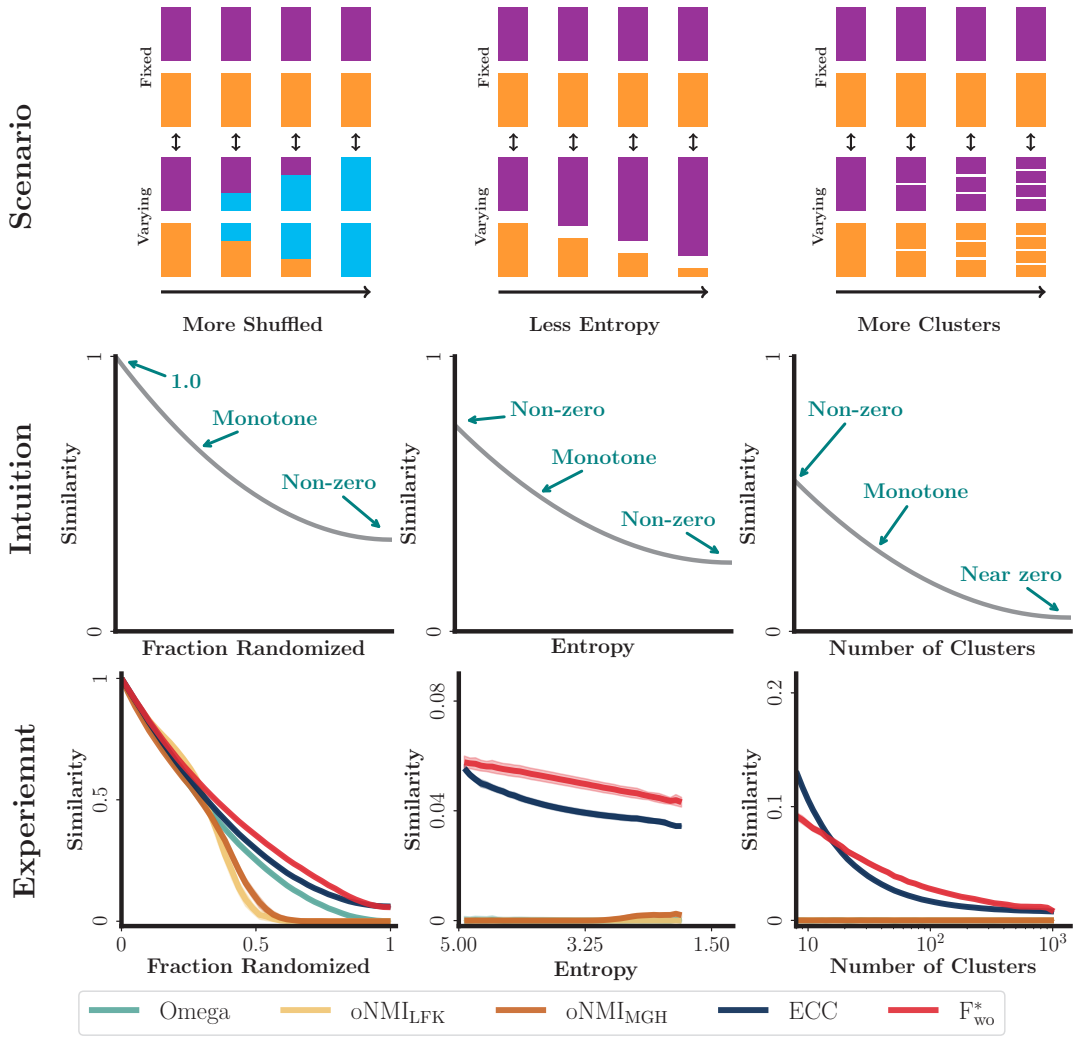


Figure 3.1: Recreation of the experiment in Figure 2 of [27] with F^*_{wo}. Each column represents one scenario, including a visual of the scenario, the intuitive behaviour of a similarity measure in this scenario, and the actual behaviour of several measures in each row respectively. Each line is the average of 100 simulations and the shaded region (usually too small to appear) covers plus or minus one standard deviation. In each scenario, the proposed F^*_{wo} measure matches our intuition.

3.4.2 Graph Aware Clustering Comparison

Since overlapping clusters are often necessary to fully capture the communities in complex networks [36], we provide a short discussion about incorporating the structure of the

graph when comparing graph clusterings. In particular, Poulin and Théberge [54] propose several graph-aware extensions of popular partition measures by transforming a partition of the vertices to a binary edge classification (inter-part versus intra-part). In the case of overlapping clusters, we cannot use a binary edge classification, but we can create an edge clustering from a node clustering. For a graph G with vertices V and edges E, and a clustering $\mathcal{C}$ of the vertices, we define the edge clustering induced by $\mathcal{C}$ as

$$\mathcal{E}_G(\mathcal{C}) = \{\{uv \in E : u, v \in C\} \,\forall\, C \in \mathcal{C}\}. \tag{3.17}$$

We can also define the vertex clustering induced by an edge clustering, as is commonly used by edge clustering algorithms [1],

$$\mathcal{C}_G(\mathcal{E}) = \{\{v \in V : \exists\, uv \in C\} \,\forall\, C \in \mathcal{E}\}. \tag{3.18}$$

When evaluating the performance of a graph clustering algorithm, we can compare clusterings from both the vertex and edge perspectives. However, as could be the case when evaluating a linkage based edge clustering algorithm [1, 20], an edge cluster may span a vertex cluster while only covering a small fraction of the internal edges. To remedy this, we can define the closure of an edge clustering $\bar{\mathcal{E}}_G$ to add these internal edges back to the edge cluster, which can be computed as $\bar{\mathcal{E}}_G = \mathcal{E}_G(\mathcal{C}_G(\mathcal{E}))$. When using a graph-aware measure for comparing an edge clustering algorithm to a ground truth vertex clustering, we recommend using the closed edge clustering since the internal edges will be included in the edge clustering induced by the ground truth.

In Figure 3.3, we evaluate three resolutions of the Leiden [60] graph clustering algorithm on synthetic ABCD [35] graphs in an experiment similar to that in Figure 5 of [54]. The different resolutions produce different cluster granularities, with 3.75 giving a fine clustering (many small clusters), 0.5 producing a coarse clustering (few large clusters), and the default resolution of 1.0 producing clustering with medium-sized clusters. While

we see the same ranking from both vertex and edge perspectives for low noise (up to $\xi \approx 0.5$), the rankings are inverted for very high ξ. In a very noisy graph, most of the edges are inter-cluster, and thus the ground truth edge clustering has many outliers. Any fine partition will also have mostly inter-cluster edges, so it is similar to the ground truth from an edge clustering perspective, even if the vertex clusterings are not similar. For evaluating graph clustering algorithms, we recommend analyzing performance from both perspectives to strengthen the conclusions.

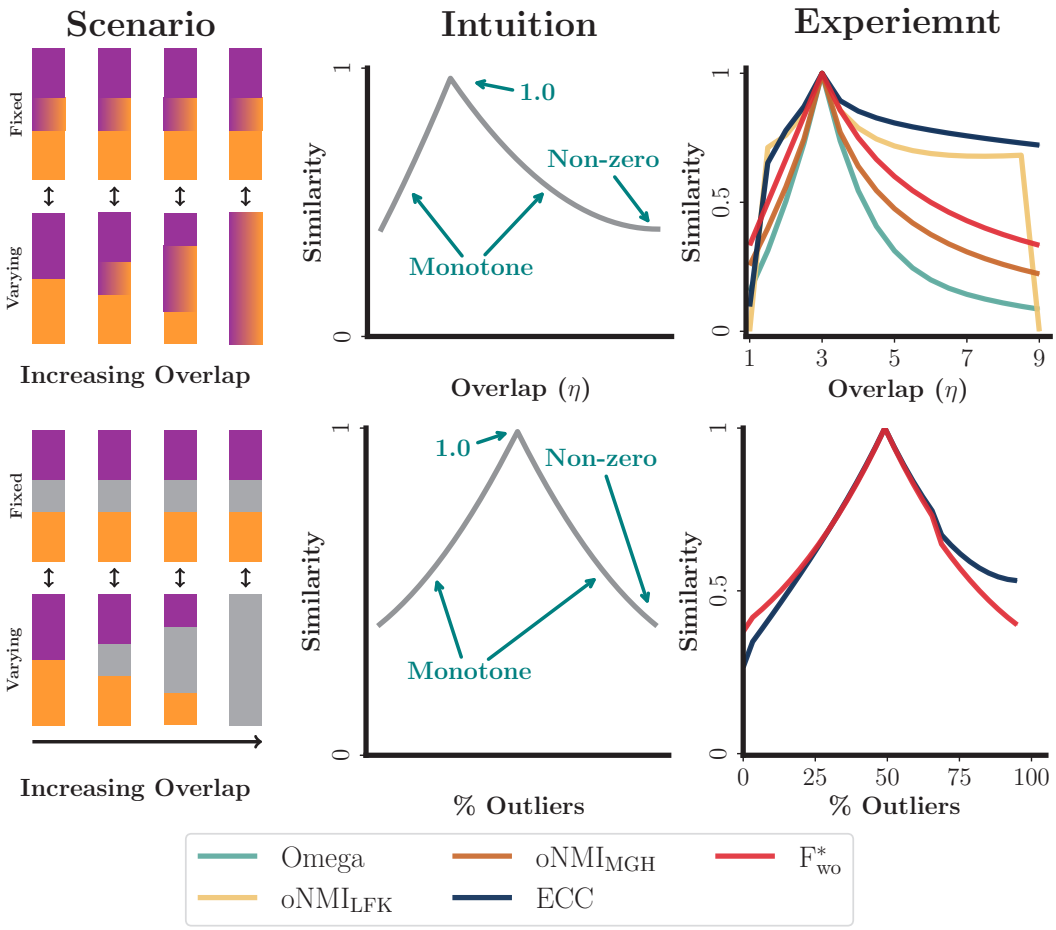


Figure 3.2: Comparing comparison measure behaviour on intuitive scenarios when overlaps or outliers are present. The top row is the scenario with overlapping clusters, and the second row has outliers; the columns, from left to right, show a visual of the scenario, the intuitive behaviour, and the actual behaviour of each method.

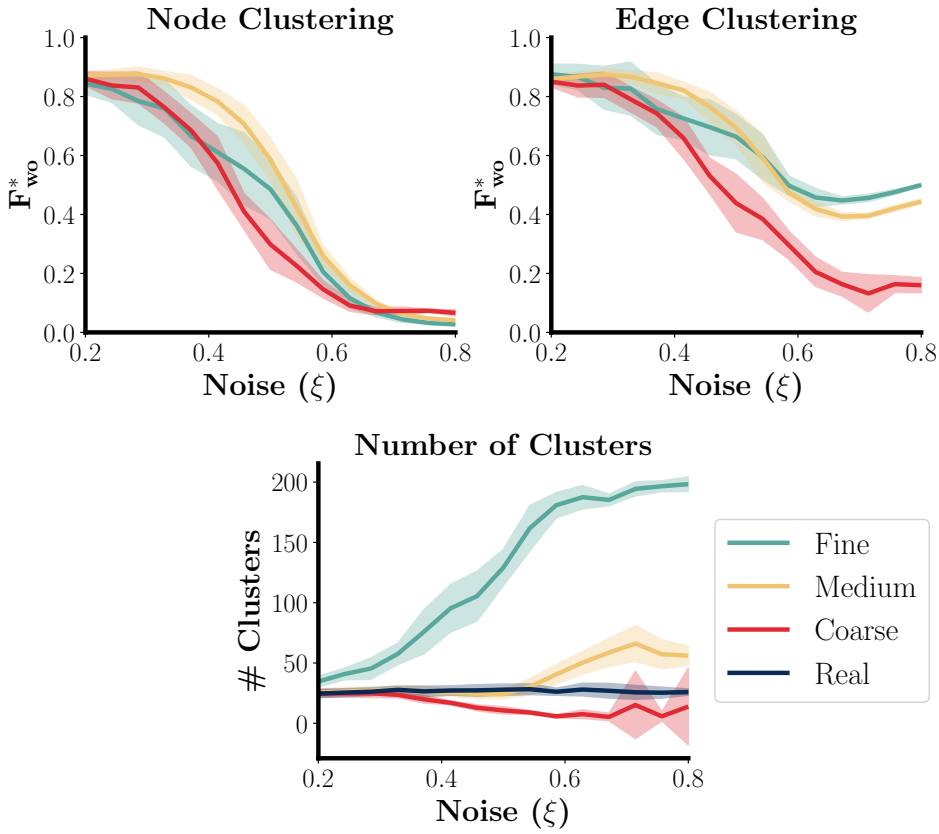


Figure 3.3: An experiment comparing graph aware clustering comparison, similar to that of Figure 5 from [54]. We use three resolutions (0.5, 1.0, and 3.75 shown in red, green, and yellow respectively) of the Leiden clustering algorithm on ABCD graphs with 2000 vertices and show, from left to right, the similarity of each detected vertex clustering to the ground truth vertex clusters, the similarity of the detected induced edge clustering to the ground truth induced edge clusters, and the number of clusters The solid line represents the average over 5 samples and the shaded region covers plus or minus one standard deviation.

Chapter 4

Benchmarking Community Detection Algorithms

In this chapter, we use the tools developed in Chapters 2 and 3 in an extrinsic community detection benchmark. First, we briefly describe three algorithms for community detection with overlaps and outliers. We refer to [25] or [64] for a comprehensive review of community detection.

4.1 Community Detection Algorithms

4.1.1 Egosplit + CAS

The EgoSplitting paradigm [24] creates an auxiliary graph G' with one or more copies of each vertex in G on which we run a standard partitioning algorithm. Define the egonet of a vertex v as the subgraph induced by the neighborhood of v, written $G[N(v)]$ (note that v is not in its egonet). The EgoSplit algorithm proceeds as follows:

1. For each $v \in V(G)$, partition the egonet of v with a community detection algorithm. Let $P_v = \{P_1, P_2, \dots P_{t(v)}\}$ be the resulting partition, where $t(v) = |P_v|$ and the

index order is arbitrary. For $u \in N(v)$, let $\ell(u, v) = i$ such that $u \in P_i$.

2. Create G' as an empty graph with $\sum_{v \in V(G)} t(v)$ vertices, with each $v \in V(G)$ associated to vertices $v_1, v_2, \ldots v_{t(v)}$ in $V(G')$.

3. For each edge uv in G add a corresponding edge to G' between $u_{\ell(v,u)}$ and $v_{\ell(u,v)}$.

4. Run a community detection algorithm on G' to get communities $\mathcal{C}'$.

5. Create the final communities

$$\mathcal{C} = \{\{v \in V(G) : \{v_1, v_2, \ldots v_{t(v)}\} \cap C' \neq \emptyset\} : C' \in \mathcal{C}'\}. \tag{4.1}$$

For community detection on the egonets, we use Label Propagation [55], and for community detection on G' we use ECG [53]. As described in [24], we also drop all communities of size less than 5.

While the egosplitting framework allows for overlapping communities (and possibly a few outliers after dropping small communities), since the ECG algorithm does not allow outliers, some vertices may be very weakly attached to their communities. In a recent paper [6], we propose post-processing a clustering using a Community Association Score (CAS). The best performing score is the Normalized Internal Edge Fraction (NIEF), defined as

$$\text{NIEF}(v, C) := \max\left\{\text{IEF}(v, C) - \frac{vol(C)}{vol(V)}, 0\right\}, \tag{4.2}$$

and motivated as the internal edge fraction corrected for community size. A simple post-processing algorithm is to accept a user-defined threshold τ, and add a vertex v to a community C if $\text{NIEF}(v, C) > \tau$ or remove v from C if $\text{NIEF}(v, C) < \tau$. We repeatedly perform the post-processing until the communities stablise (or we get stuck in a 2-step loop of adding then removing a few vertices, in which case we terminate). Pseudocode of the post-processing algorithm in shown in Algorithm 1. In the following experiments,

we fix $\tau = 0.5$.

4.1.2 LC + HSLC

This method combines the Link Communities (LC) edge clustering scores with Hierarchical Sinlge Linkage Clustering (HSLC) [20] to allow for outliers and overlapping communities. LC defines a similarity measure between adjacent edges by considering the similarity of their neighborhoods, stored as the weights on the line graph. The line graph LG of a graph G has vertices $V(LG) = E(G)$ and edges $E(LG) = \{ef : e \text{ and } f \text{ are adjacent in} G\}$. For adjacent edges $e = uv$ and $f = vw$, the edge similarity s in LC is the jaccard similarity of the neighborhood of the non-central vertex;

$$s(e, f) = \frac{|N(u) \cap N(w)|}{|N(u) \cup N(w)|}. \tag{4.3}$$

Next, hierarchical single linkage clustering is applied to the weighted line graph. The algorithm is taken from HDBSCAN [15, 48], a popular density based vector clustering algorithm and adapted for arbitrary similarity graphs.

First, a single-linkage dendrogram is built from the similarity graph LG. The clusters at cut level λ are the connected components of the subgraph with all vertices and edges $\{ef \in E(LG) : s(e, f) \geq \lambda\}$.

This dendrogram is generally too complex to visualize (consider that when $\lambda \to \infty$ every object has its own cluster), so to condense the dendrogram so that it only tracks significant clusters, a parameter m_s is introduced to control the minimum cluster size. When traversing the dendrogram top-down, if a cluster is split, then one of the following three cases will occur:

1. The cluster splits into several clusters, each with less than m_s vertices. We say the cluster has disappeared.

Algorithm 1 CAS Post-processing Algorithm

Ensure: $\mathcal{C} = Array[0 : |V| - 1]\ with\ C[v] = \{i : v \in C_i\}$

```
function PostProcess(C, G, τ)
    last2round = {}
    lastround = {}
    done ← False
    round = 1
    while not done and round ≤ 100 do
        C' ← Array[0 : |V| − 1] of {}
        done ← True
        for v ∈ V do
            potentialcoms ← C[v]
            for u ∈ N(v) do
                potentialcoms ← potentialcoms ∪ C[u]
            end for
            for c ∈ potentialcoms do
                cas ← CAS(v, c, G)
                if cas ≥ τ then
                    C'[v] ← C'[v] ∪ {c}
                    done ← False
                end if
            end for
        end for
        thisround = {v ∈ V : C[v] ≠ C'[v]}
        if thisround = last2round then
            done = True
        end if
        last2round ← lastround
        lastround ← thisround
        C ← C'
        round ← round + 1
    end while
return C
end function
```

2. The cluster splits into several clusters, at least two of which have at least m_s vertices. We say all sub-clusters with at least m_s vertices are significant and different from the original.

3. The cluster splits into one cluster with at least m_s vertices and one or more clusters with less than m_s vertices. We say the original cluster shrinks, and the largest sub-cluster retains the name of the original cluster. The other clusters become outliers.

The condensed dendrogram will have far fewer clusters, and naturally defines a cut level at which every cluster will disappear (for $m_s > 1$).

Finally, even though the condensed dendrogram is practical for investigation, many applications still require a single set of clusters. HDBSCAN [15] defines a persistence score for each cluster, and then provides an algorithm to find the set of non-overlapping clusters to maximize total persistence. For a cluster $C_i \subseteq K$, define the death of the cluster as $\lambda_{min}(C_i) = \min\{\lambda : C_i \text{ exists}\}$. Also, define the contribution of each object $k_j \in C_i$ as $\lambda_{max}(k_j, C_i) = \max\{\lambda : k_j \in C_i\}$. Then, the persistence of cluster C_i is given by the equation

$$\sigma(C_i) = \sum_{k_j \in C_i} \left(\lambda_{max}(k_j, C_i) - \lambda_{min}(C_i)\right). \tag{4.4}$$

The optimal (and returned) flat clustering is the set of clusters $\mathcal{C}$ that maximizes $\sum_{C_i \in \mathcal{C}} \sigma(C_i)$ subject to $C_i \cap C_j = \emptyset \; \forall \; C_i, C_j \in \mathcal{C}$.

Since we run HSLC on LG, it returns a set of edge clusters. We project the edge cluster to the vertices by including a vertex in a cluster if it is incident to at least one edge in that cluster (recall Equation 3.18).

4.1.3 OSLOM

The Order Statistics Local Optimization Method (OSLOM) clustering algorithm [44] is a complicated method that uses a statistical significance [43] based CAS score to create

a local clustering [3] algorithm. Their CAS score is derived from the probability that a vertex not in a community would have at least the same number of connections to the community if the graph was rewired following the configuration model. This score is used to develop a local clustering algorithm that finds an optimially significant community containing a given seed vertex. Many such communities are detected and combined or removed (if they overlap too much) according to several significance tests and preset thresholds. We refer to [44] for a detailed explanation of the algorithm and motivations, and [45] for the code used in the following experiments.

4.2 Synthetic Data

For a comprehensive benchmark where we trust the ground truth, we use a large collection of graphs from the ABCD+o^2 model. We vary $\xi \in [0.2, 0.6]$ and $\eta \in [1, 3]$, and fix $n = 10000, s_0 = 1000, \delta = 5, \Delta = 1000, \gamma = 2.5, s = 25, S = 1000, \beta = 1.5, d = 8$, and $\rho = 0.25$. For each set of parameters, we generate 10 sample graphs and report the mean F^*_{wo} of the detected clustering compared to the ground truth.

Figure 4.1 shows the performance of each algorithm as a heatmap. As expected, each algorithm performs quite well when both ξ and η are small (especially when $\eta = 1$ and there is no overlap), but performance tends to decrease as either parameter increases. The exception is when comparing the induced edge clusterings for EGOSPLIT+CAS and OSLOM, in which case the performance tends to increase as ξ increases, indicating these algorithms find small clusters or lots of outliers.

A heatmap gives a comprehensive overview of one algorithm, but it can be difficult to quickly compare the algorithms. To facilitate a quick comparison, Figure 4.2 plots all three algorithms on one graph by fixing one of ξ or η. We find that no one algorithm is always best, but the OSLOM algorithm does perform better for small ξ or small η.

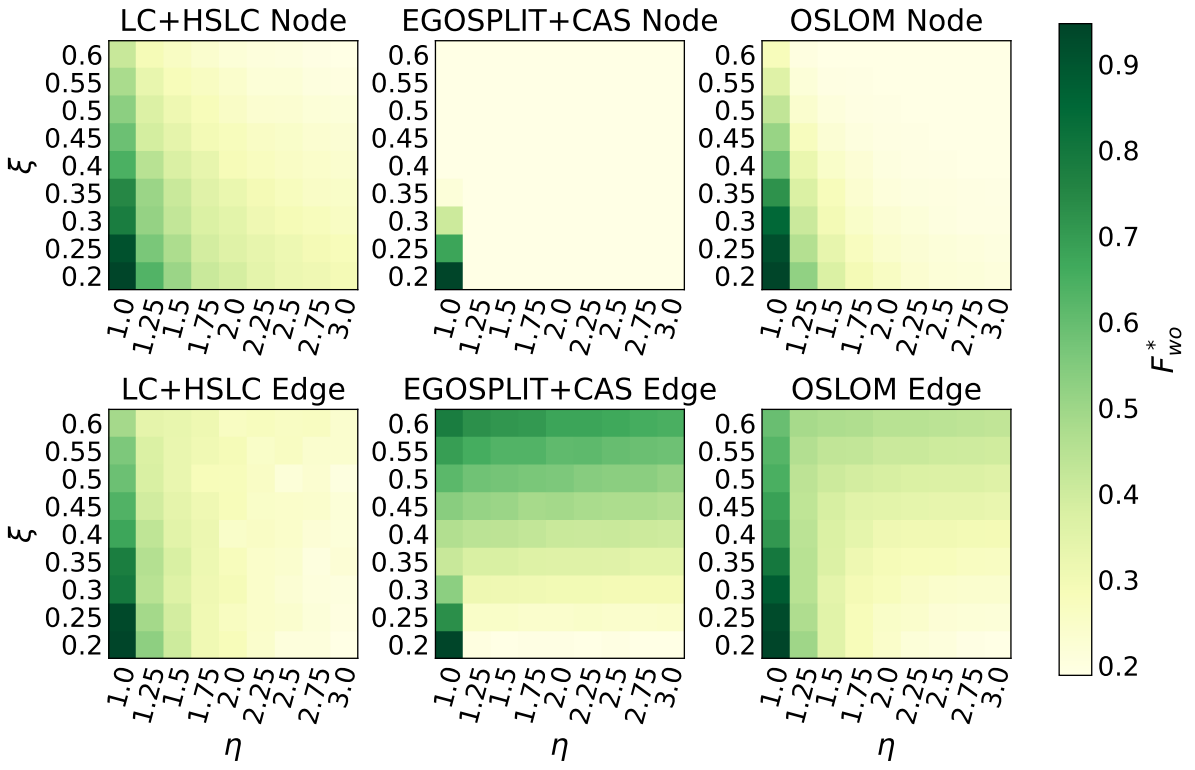


Figure 4.1: Evaluating the performance of each algorithm with respect to variable ξ and η.

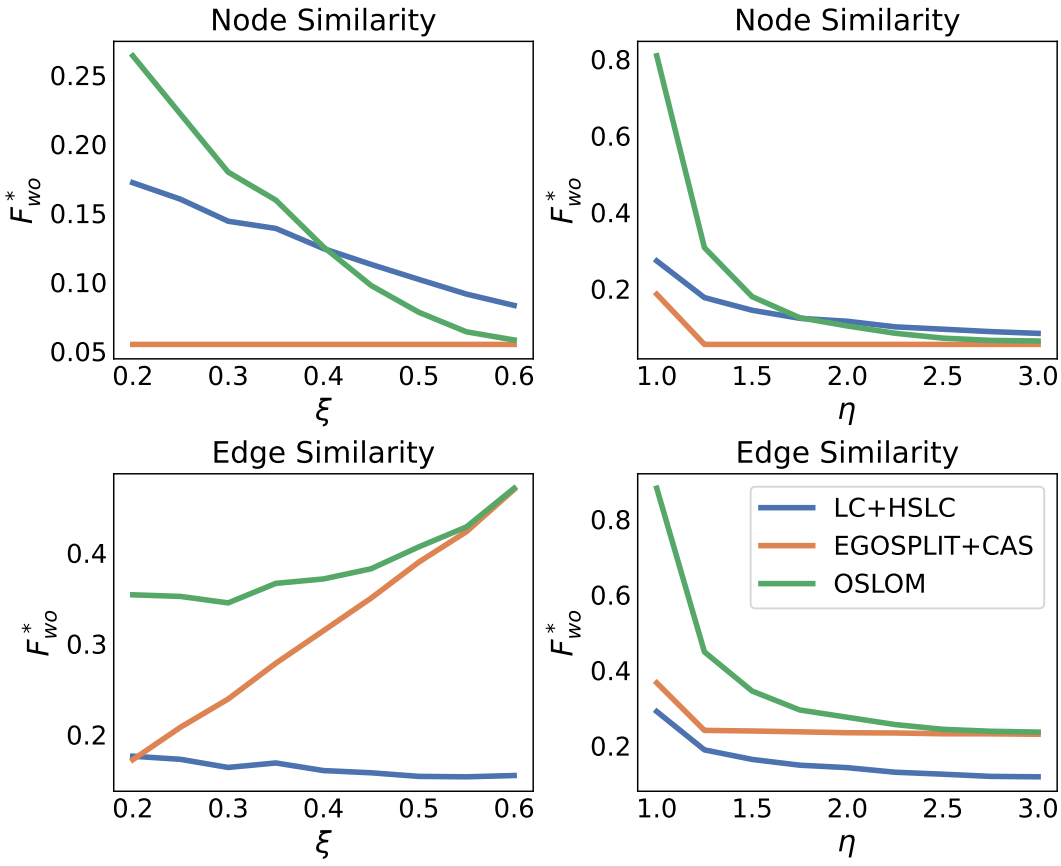


Figure 4.2: Evaluating the performance of each clustering algorithm with fixed $\eta = 1.5$ (left column) or $\xi = 0.3$ (right column).

4.3 Real Data

For real graph data with known ground truth communities, we use the DBLP, Amazon, and Youtube graphs from the SNAP graph repository [46] that were introduced in Section 2.2.2. Statistics about the graphs can be found in Table 2.2, except we use the full Youtube graph consisting of an additional 1092520 outlier vertices for a total $n = 1134890$. We were unable to run LC+HSLC on the full Youtube graph since the linegraph has over one billion edges.

In our first experiment (Figure 4.3), we run each community detection algorithm on each graph and use F^*_{wo} to compare the detected clusters with the ground truth communities. None of the algorithms perform well on the DBLP or Amazon graphs, and EGOSPLIT+CAS performs well on Youtube.

However, [65] found that most of the ground truth communities have very low intrinsic quality, and would be unlikely to be detected even (or especially) by a good clustering algorithm. Intrinsic community quality is a type of function that attempts to quantify how good a community is looking only at the structure of the graph. There are many particular functions, we refer to [65] for a review, but they found that each of these graph have about 5000 communities that score highly on a variety of intrinsic measures. They have published alternate files for the ground truth communities that consist only of those with the top 5000 intrinsic measures.

As a second experiment, we test the ability of each community detection algorithm to successfully find these high quality communities. We evaluate the detected communities $\mathcal{C}$ with the top 5000 ground truth communities $\hat{\mathcal{C}}$ using $\hat{F}^*_w(\hat{\mathcal{C}}, \mathcal{C})$. Each of the high quality ground truth communities will look for a match in the detected communities, but we do not insist that each detected community is in the ground truth, nor that all the other vertices are outliers. The results of this experiment are shown in Figure 4.4.

In this experiment, the EGOSPLIT+CAS algorithm performs the best on all three

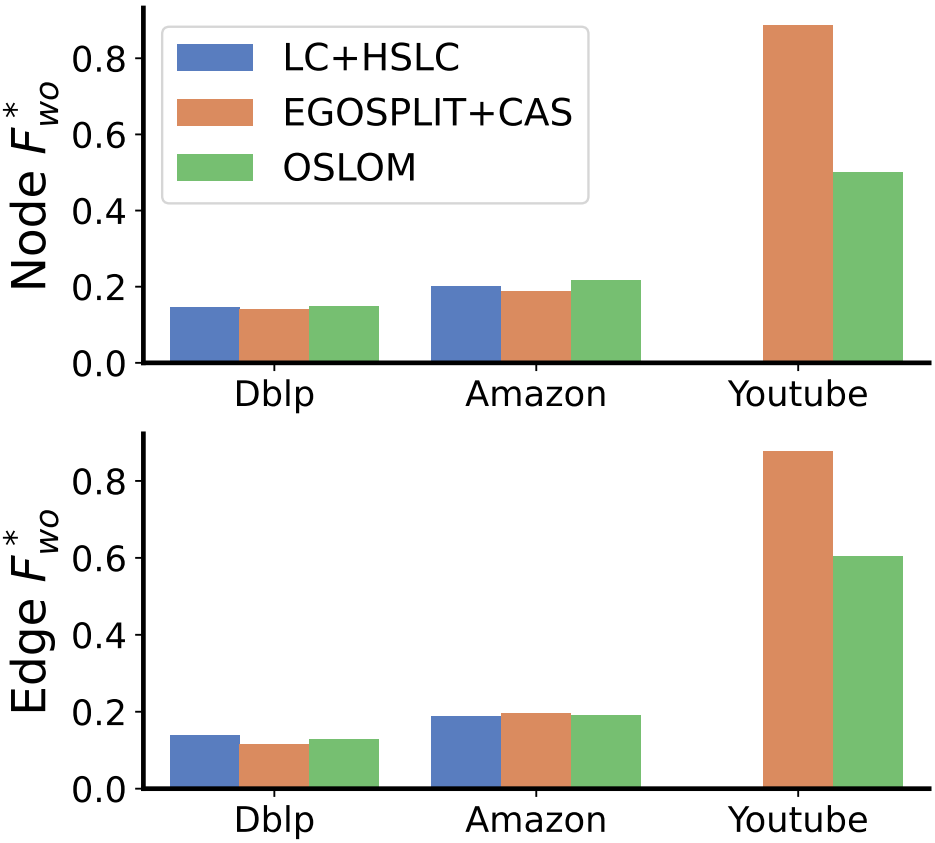


Figure 4.3: Ability of each community detection algorithm to match the full ground truth clustering on real graph datasets.

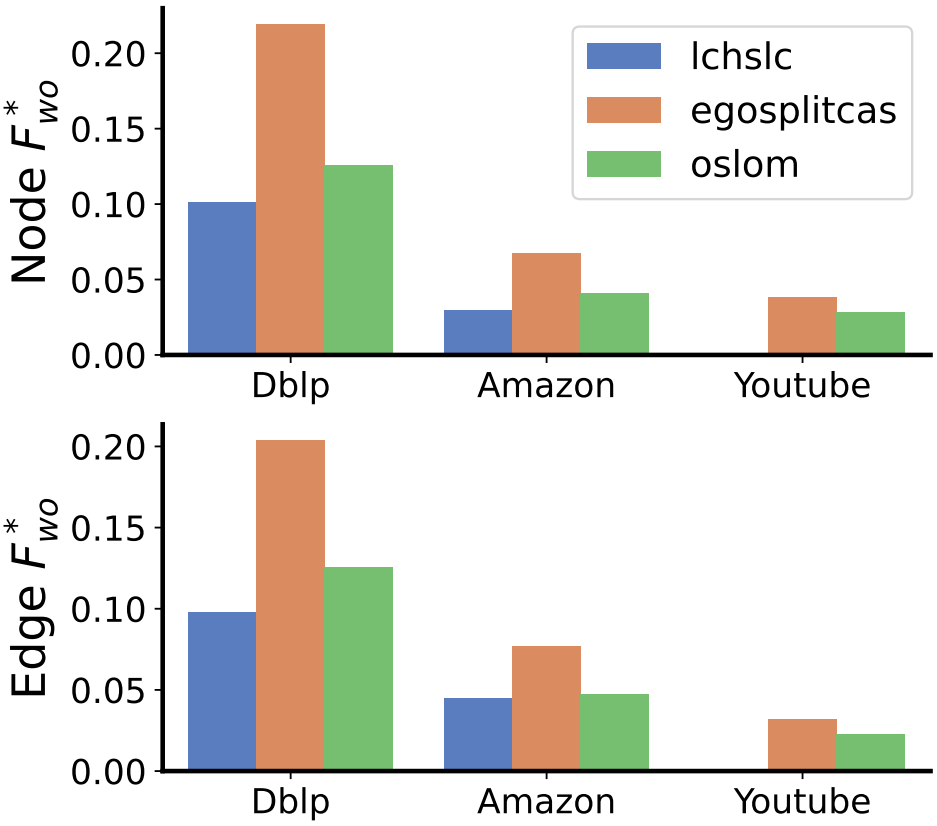


Figure 4.4: Ability of each clustering algorithm to find the top 5000 ground truth communities on real graph datasets.

graphs and for both node and edge based comparisons. The scores are still quite poor (less than 0.25), which means the high scores on the Youtube graph in the previous experiment were due to correctly finding outliers. We draw a tentative conclusion that EGOSPLIT+CAS is performing the best on the real datasets, and note this is in contrast to the inconclusive results from the synthetic experiments. However, none of the algorithms are performing very well, and there is lots of room for improvement.

Chapter 5

Conclusion

In this thesis, we developed two tools that when combined, allow for extrinsic benchmarking of community detection algorithms that may have overlaps and outliers. First, the ABCD+o^2 model provides a synthetic playground that can recreate an overlap structure similar to those seen in real graphs and allows for a smooth increase in detection difficulty. Next, we proposed a clustering comparison measure that is well-defined for clustering with overlaps and outliers, and checked that it behaves intuitively. Finally, we used these newly developed tools to evaluate three community detection methods. We hope these tools encourage further development of community detection algorithms that go beyond partitions of the vertices.

Appendix A

Additional ABCD+o^2 Figures

A.1 Density of Intersection Figures

Figures A.1, A.2 and A.3, together with the previously shown Figure 2.6, present the full experiment comparing community and intersection densities for various ρ. The results are similar to those found in Section 2.2.2; the density of individual communities in unaffected by ρ, and smaller communities are typically denser than larger communities. However, the density of the intersection does increase with ρ, meaning that dense community intersections may be caused by a positive correlation between degree and the number of communities.

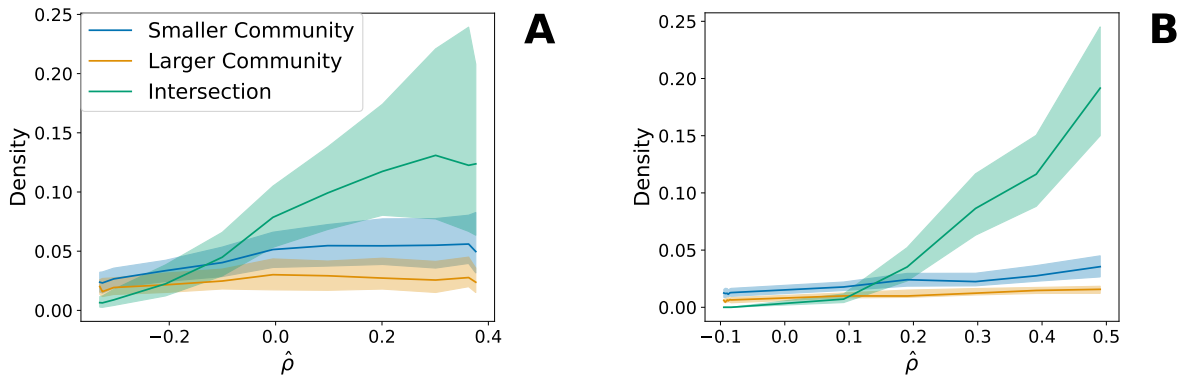


Figure A.1: The density of overlaps compared to the empirical ρ on YouTube-like ABCD+o^2 graphs with reference layer dimensions $d = 2$ (panel A) and $d = 64$ (panel B).

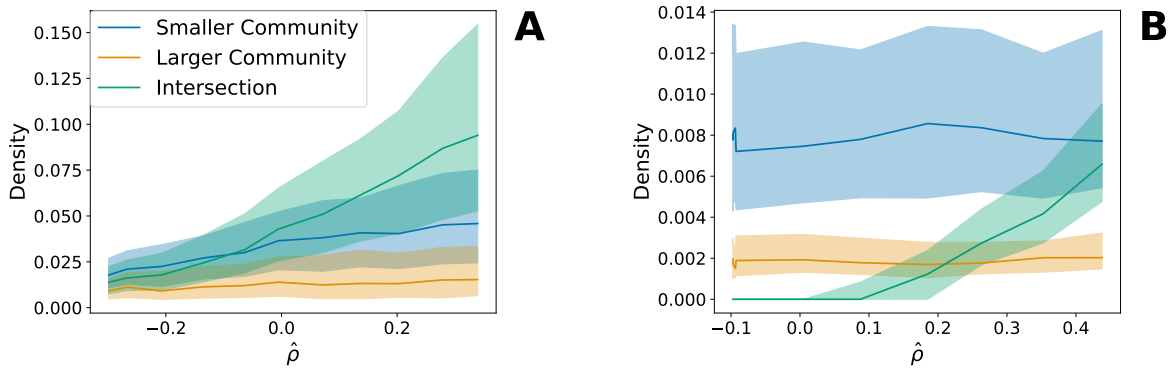


Figure A.2: The density of overlaps compared to the empirical ρ on DBLP-like ABCD+o^2 graphs with reference layer dimensions $d = 2$ (panel A) and $d = 64$ (panel B).

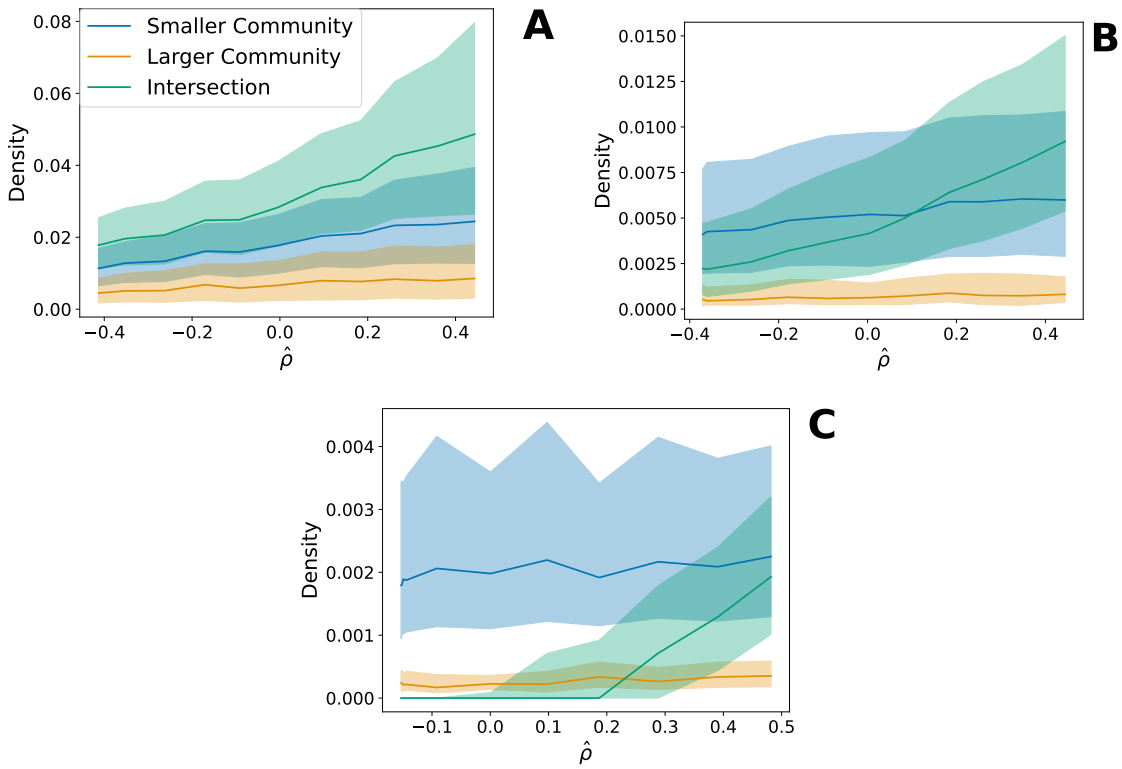


Figure A.3: The density of overlaps compared to the empirical ρ on Amazon-like ABCD+o^2 graphs with reference layer dimensions $d = 2$ (panel A), $d = 8$ (panel B) and $d = 64$ (panel C).

A.2 Internal Edge Fraction Figures

Figures A.4, A.5 and A.6, together with the previously shown Figure 2.8, present the full experiment showing the range of the top five internal edge fractions, sorted in descending order, on the real networks (left) and the corresponding ABCD+o^2 graphs (right), binned by the number of communities the node v is in. The results are similar to those found in Section 2.2.2, with a node in ℓ communities typically having ℓ large but unequal IEF values. Furthermore, the gap between the ℓ and $\ell + 1$ IEF values tends to decrease as ℓ

increases.

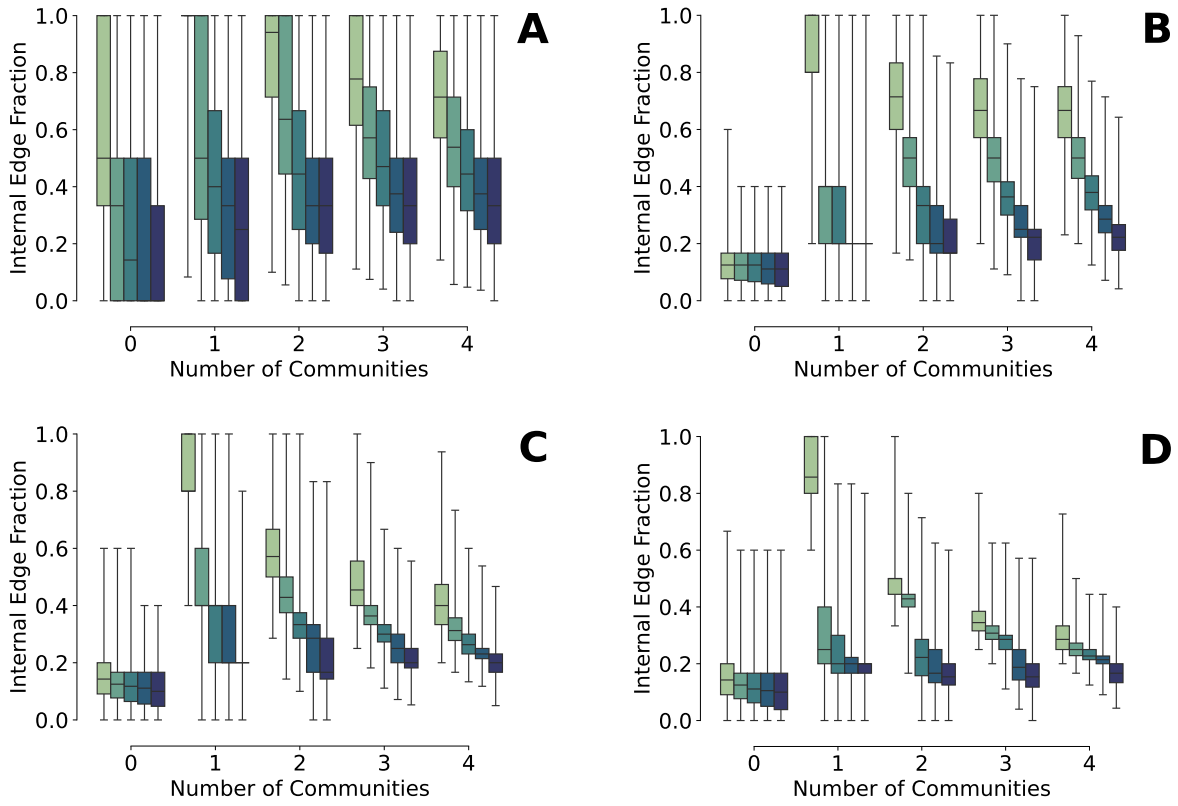


Figure A.4: The top 5 IEF values in decreasing order, grouped by the number of communities the node belongs to, for DBLP and DBLP-like ABCD+o^{2}graphs. Panel A corresponds to the real graph, and panels B, C, D correspond to the synthetic counterparts with reference layer dimension $d = 2$, $d = 8$, and $d = 64$ respectively. Panels A and C correspond to panels A and B of Figure 2.8, but are repeated here for easy comparison with the other reference layer dimensions.

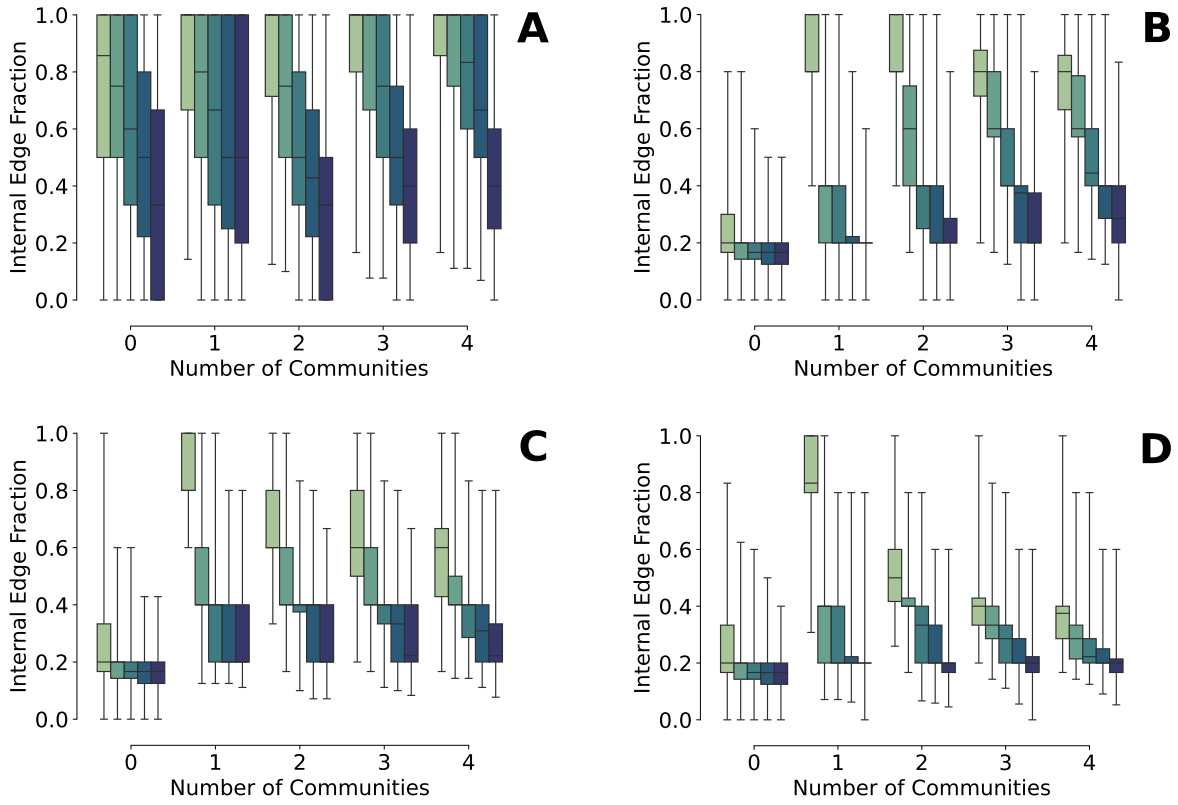


Figure A.5: The top 5 IEF values in decreasing order, grouped by the number of communities the node belongs to, for Amazon and Amazon-like ABCD+o^{2}graphs. Panel A corresponds to the real graph, and panels B, C, D correspond to the synthetic counterparts with reference layer dimension $d = 2$, $d = 8$, and $d = 64$ respectively

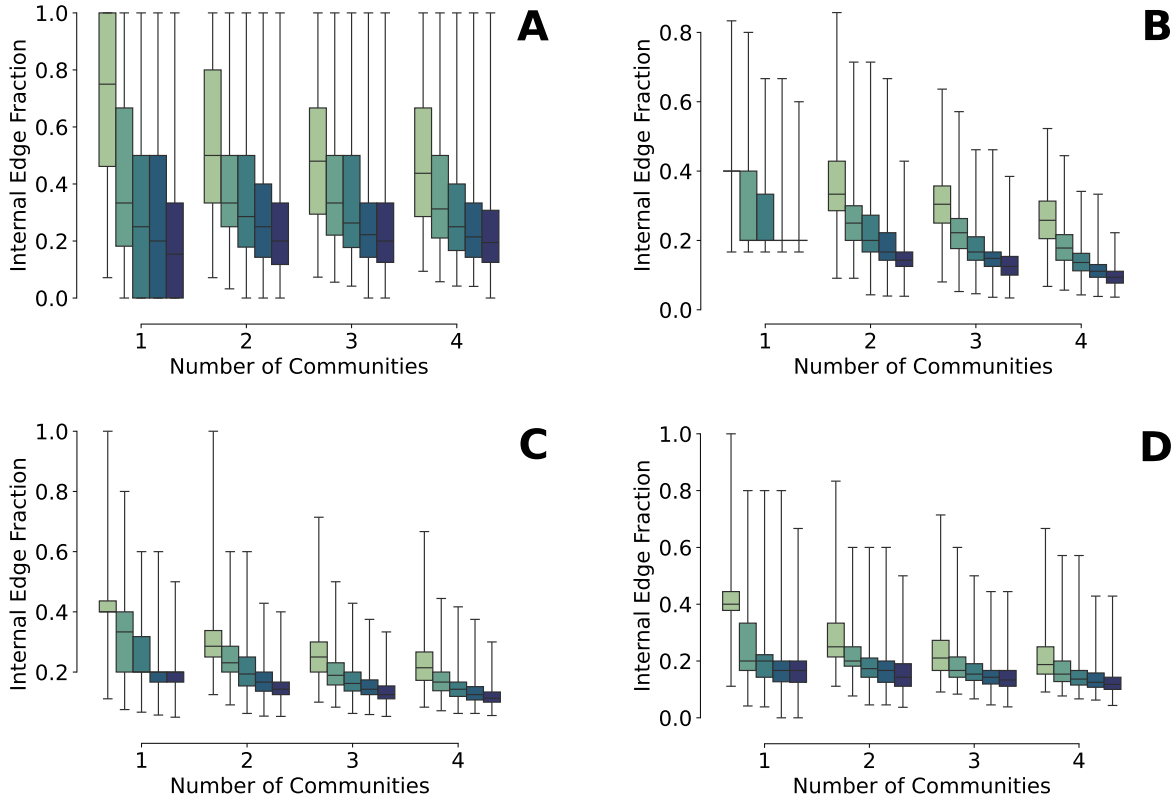


Figure A.6: The top 5 IEF values in decreasing order, grouped by the number of communities the node belongs to, for YouTube and YouTube-like ABCD+o^{2}graphs. Panel A corresponds to the real graph, and panels B, C, D correspond to the synthetic counterparts with reference layer dimension $d = 2$, $d = 8$, and $d = 64$ respectively

Appendix B

Python Code for Comparing Clusterings

```
import numpy as np
import scipy.sparse as sp


def compare(c1, c2):
    # Inputs c1, c2: The clusterings to compare.
    # Both must be scipy csr matrices of size (n_clusters,
       n_objects)
    # with c[i,j] = 1 if object j is in cluster i.
    if c1.shape[0] == 0 and c2.shape[0] == 0: # No clusters
        return 1
    c1_sizes = np.asarray(c1.sum(axis=1)).reshape(-1)
    c1_props = c1_sizes / np.sum(c1_sizes)
    c2_sizes = np.asarray(c2.sum(axis=1)).reshape(-1)
    c2_props = c2_sizes / np.sum(c2_sizes)
```

```
intersect = c1.astype("float64") @ c2.transpose().astype("
   float64")
row_sizes = sp.diags(c1_sizes)
col_sizes = sp.diags(c2_sizes)
nz = intersect.copy() # Store non-zero data indices
nz.data[:] = 1
union = row_sizes @ nz + nz @ col_sizes - intersect
union.data = 1 / union.data
fs = intersect.multiply(union)
fs_l = fs.max(axis=1).toarray().reshape(-1)
fs_lw = np.sum(fs_l * c1_props)
fs_r = fs.max(axis=0).toarray().reshape(-1)
fs_rw = np.sum(fs_r * c2_props)
c1_o = c1.count_nonzero(axis=0) == 0
c2_o = c2.count_nonzero(axis=0) == 0
o_intersect = np.sum(c1_o * c2_o)
o_fs = 0
if o_intersect > 0:
    o_fs = o_intersect / (np.sum(c1_o) + np.sum(c2_o) -
       o_intersect)
alpha = 1 - np.sum(c1_o) / len(c1_o)
beta = 1 - np.sum(c2_o) / len(c2_o)
fs_l_wo = (1 - alpha) * o_fs + alpha * fs_lw
fs_r_wo = (1 - beta) * o_fs + beta * fs_rw
return 0.5 * (fs_l_wo + fs_r_wo)
```

References

[1] Ahn, Y.-Y., Bagrow, J. P., and Lehmann, S. Link communities reveal multiscale complexity in networks. *Nature 466* (2010), 761–764.

[2] Alstott, J., Bullmore, E., and Plenz, D. powerlaw: A python package for analysis of heavy-tailed distributions. *PLoS ONE 9*, 1 (2014).

[3] Andersen, R., Chung, F., and Lang, K. Local graph partitioning using pagerank vectors. In *47th Annual IEEE Symposium on Foundations of Computer Science (FOCS'06)* (2006), pp. 475–486.

[4] Aref, S., Mostajabdaveh, M., and Chheda, H. Bayan algorithm: Detecting communities in networks through exact and approximate optimization of modularity. *Phys. Rev. E 110* (2024), 044315.

[5] Barabási, A.-L. *Network science*. Cambridge University Press, 2016.

[6] Barrett, J., DeWolfe, R., Kamiński, B., Prałat, P., Smith, A., and Théberge, F. Improving community detection via community association strength scores. In *Modelling and Mining Networks* (Cham, 2025), Springer Nature Switzerland, pp. 81–95.

[7] Barrett, J., DeWolfe, R., Kamiński, B., Prałat, P., Smith, A., and Théberge, F. The artificial benchmark for community detection with outliers

and overlapping communities (abcd+o^2). arXiv:2506.05486 [social and information networks].

[8] Barrett, J., DeWolfe, R., Kamiński, B., Prałat, P., Smith, A., and Théberge, F. The artificial benchmark for community detection with outliers and overlapping communities (ABCD+o^2). In *Modelling and Mining Networks* (2025), vol. 15669 of *Lecture Notes in Computer Science*, pp. 125–140.

[9] Barrett, J., Kamiński, B., Prałat, P., and Théberge, F. Self-similarity of communities of the ABCD model. *Theoretical Computer Science 1026* (2025), 115012.

[10] Bender, E. A., and Canfield, E. R. The asymptotic number of labeled graphs with given degree sequences. *Journal of Combinatorial Theory, Series A 24*, 3 (1978), 296–307.

[11] Bentley, J. L. Multidimensional binary search trees used for associative searching. *Commun. ACM 18*, 9 (Sept. 1975), 509–517.

[12] Blondel, V. D., Guillaume, J.-L., Lambiotte, R., and Lefebvre, E. Fast unfolding of communities in large networks. *Journal of Statistical Mechanics: Theory and Experiment 2008*, 10 (2008).

[13] Boguna, M., Bonamassa, I., De Domenico, M., Havlin, S., Krioukov, D., and Serrano, M. Á. Network geometry. *Nature Reviews Physics 3*, 2 (2021), 114–135.

[14] Bollobás, B. A probabilistic proof of an asymptotic formula for the number of labelled regular graphs. *European Journal of Combinatorics 1*, 4 (1980), 311–316.

[15] Campello, R. J. G. B., Moulavi, D., and Sander, J. Density-based clustering based on hierarchical density estimates. In *Advances in Knowledge Discovery and*

Data Mining (2013), vol. 7819 of *Lecture Notes in Computer Science*, Springer, Berlin, Heidelberg, pp. 160–172.

[16] Christen, P., Hand, D. J., and Kirielle, N. A review of the F-measure: Its history, properties, criticism, and alternatives. *ACM Computing Surveys 56*, 3 (2023).

[17] Chykhradze, K., Korshunov, A., Buzun, N., Pastukhov, R., Kuzyurin, N., Turdakov, D., and Kim, H. Distributed generation of billion-node social graphs with overlapping community structure. In *Complex Networks V* (Cham, 2014), P. Contucci, R. Menezes, A. Omicini, and J. Poncela-Casasnovas, Eds., Springer International Publishing, pp. 199–208.

[18] Clauset, A., Shalizi, C. R., and Newman, M. E. J. Power-law distributions in empirical data. *SIAM Review 51*, 4 (2009), 661–703.

[19] Collins, L. M., and Dent, C. W. Omega: A general formulation of the rand index of cluster recovery suitable for non-disjoint solutions. *Multivariate Behavioral Research 23*, 2 (1988), 231–242.

[20] DeWolfe, R. Hierarchical single-linkage clustering for community detection with overlaps and outliers.

[21] DeWolfe, R. Leveraging non-linear dimension reduction and random walk co-occurrence for node embedding.

[22] DeWolfe, R., and Andrews, J. L. Random models for adjusting fuzzy rand index extensions. *Advances in Data Analysis and Classification 19* (2025), 361–385.

[23] DeWolfe, R., and Théberge, F. Detecting patterns of interaction in temporal hypergraphs via edge clustering. In *Advances in Social Networks Analysis and Mining* (2026), Lecture Notes in Social Networks, Springer Cham, pp. 271–285.

[24] Epasto, A., Lattanzi, S., and Paes Leme, R. Ego-splitting framework: From non-overlapping to overlapping clusters. In *Proceedings of the 23rd ACM SIGKDD international conference on knowledge discovery and data mining* (2017), pp. 145–154.

[25] Fortunato, S. Community detection in graphs. *Physics Reports 486*, 3-5 (2010), 75–174.

[26] Gates, A. J., and Ahn, Y.-Y. The impact of random models on clustering similarity. *Journal of Machine Learning Research 18*, 87 (2017), 1–28.

[27] Gates, A. J., Wood, I. B., Hetrick, W. P., and Ahn, Y.-Y. Element-centric clustering comparison unifies overlaps and hierarchy. *Scientific Reports 9*, 1 (2019), 8574.

[28] Girvan, M., and Newman, M. E. J. Community structure in social and biological networks. *Proceedings of the National Academy of Sciences 99*, 12 (2002), 7821–7826.

[29] Gösgens, M. M., Tikhonov, A., and Prokhorenkova, L. Systematic analysis of cluster similarity indices: How to validate validation measures. In *International Conference on Machine Learning* (2021), vol. 139 of *PMLR*, pp. 3799–3808.

[30] Hand, D. J., Christen, P., and Kirielle, N. F*: an interpretable transformation of the F-measure. *Machine Learning 110*, 3 (2021), 451–456.

[31] Hennig, C., Meila, M., Murtagh, F., and Rocci, R. *Handbook of Cluster Analysis*. Chapman and Hall/CRC, New York, 2015.

[32] Henry, A. D., Prałat, P., and Zhang, C.-Q. Emergence of segregation in evolving social networks. *Proceedings of the National Academy of Sciences 108*, 21 (2011), 8605–8610.

[33] Hubert, L., and Arabie, P. Comparing partitions. *Journal of Classification 2*, 1 (1985), 193–218.

[34] Kamiński, B., Pankratz, B., Prałat, P., and Théberge, F. Modularity of the ABCD random graph model with community structure. *Journal of Complex Networks 10*, 6 (2022), cnac050.

[35] Kamiński, B., Prałat, P., and Théberge, F. Artificial benchmark for community detection (ABCD)—fast random graph model with community structure. *Network Science* (2021), 1–26.

[36] Kaminski, B., Prałat, P., and Théberge, F. *Mining complex networks*. Chapman and Hall/CRC, 2021.

[37] Kamiński, B., Prałat, P., and Théberge, F. Hypergraph artificial benchmark for community detection (h–ABCD). *Journal of Complex Networks 11*, 4 (2023), cnad028.

[38] Kamiński, B., Prałat, P., and Théberge, F. Artificial benchmark for community detection with outliers (abcd+o). *Applied Network Science 8*, 1 (2023), 25.

[39] Kvalseth, T. O. Entropy and correlation: Some comments. *IEEE Transactions on Systems, Man, and Cybernetics 17*, 3 (1987), 517–519.

[40] Lancichinetti, A., and Fortunato, S. Benchmarks for testing community detection algorithms on directed and weighted graphs with overlapping communities. *Physical Review E 80*, 1 (2009), 016118.

[41] Lancichinetti, A., Fortunato, S., and Kertész, J. Detecting the overlapping and hierarchical community structure in complex networks. *New Journal of Physics 11* (2009), 033015.

[42] Lancichinetti, A., Fortunato, S., and Radicchi, F. Benchmark graphs for testing community detection algorithms. *Physical review E 78*, 4 (2008), 046110.

[43] Lancichinetti, A., Radicchi, F., and Ramasco, J. J. Statistical significance of communities in networks. *Phys. Rev. E 81* (Apr 2010), 046110.

[44] Lancichinetti, A., Radicchi, F., Ramasco, J. J., and Fortunato, S. Finding statistically significant communities in networks. *PLoS ONE 6*, 4 (2011), e18961.

[45] Lancichinetti, A., Radicchi, F., Ramasco, J. J., and Fortunato, S. Order statistics local optimization method, 2011.

[46] Leskovec, J., and Krevl, A. SNAP Datasets: Stanford large network dataset collection, June 2014.

[47] McDaid, A. F., Greene, D., and Hurley, N. Normalized mutual information to evaluate overlapping community finding algorithms, 2013.

[48] McInnes, L., Healy, J., and Astels, S. hdbscan: Hierarchical density based clustering. *Journal of Open Source Software 2*, 11 (2017).

[49] Meilă, M. Comparing clusterings—an information based distance. *Journal of Multivariate Analysis 98*, 5 (2007), 873–895.

[50] Morey, L. C., and Agresti, A. The measurement of classification agreement: An adjustment to the rand statistic for chance agreement. *Educational and Psychological Measurement 44* (1984), 33–37.

[51] Orman, G. K., and Labatut, V. A comparison of community detection algorithms on artificial networks. In *Discovery Science: 12th International Conference, DS 2009, Porto, Portugal, October 3-5, 2009 12* (2009), Springer, pp. 242–256.

[52] Page, L., Brin, S., Motwani, R., and Winograd, T. The pagerank citation ranking: Bringing order to the web. Technical Report 1999-66, Stanford InfoLab, November 1999.

[53] Poulin, V., and Théberge, F. Ensemble clustering for graphs: comparisons and applications. *Applied Network Science 4*, 4 (2019).

[54] Poulin, V., and Théberge, F. Comparing graph clusterings: Set partition measures vs. graph-aware measures. *IEEE Transactions on Pattern Analysis and Machine Intelligence 43*, 6 (2021), 2127–2132.

[55] Raghavan, U. N., Albert, R., and Kumara, S. Near linear time algorithm to detect community structures in large-scale networks. *Phys. Rev. E 76* (2007), 036106.

[56] Rezaei, M., and Fränti, P. Set matching measures for external cluster validity. *IEEE Transactions on Knowledge and Data Engineering 28*, 8 (2016), 2173–2186.

[57] Sengupta, N., Hamann, M., and Wagner, D. Benchmark generator for dynamic overlapping communities in networks. In *2017 IEEE International Conference on Data Mining (ICDM)* (2017), pp. 415–424.

[58] Serrano, M. Á., and Boguñá, M. *The Shortest Path to Network Geometry: A Practical Guide to Basic Models and Applications*. Cambridge University Press, 2021.

[59] Serrano, M. Á., Krioukov, D., and Boguná, M. Self-similarity of complex networks and hidden metric spaces. *Physical review letters 100*, 7 (2008), 078701.

[60] Traag, V. A., Waltman, L., and van Eck, N. J. From Louvain to Leiden: guaranteeing well-connected communities. *Scientific Reports 9*, 1 (2019).

[61] Vinh, N. X., Epps, J., and Bailey, J. Information theoretic measures for clusterings comparison: Variants, properties, normalization and correction for chance. *Journal of Machine Learning Research 11*, 95 (2010), 2837–2854.

[62] Wormald, N. C. Generating random regular graphs. *Journal of algorithms 5*, 2 (1984), 247–280.

[63] Wormald, N. C. *Models of Random Regular Graphs.* London Mathematical Society Lecture Note Series. Cambridge University Press, 1999, pp. 239–298.

[64] Xie, J., Kelley, S., and Szymanski, B. K. Overlapping community detection in networks: The state-of-the-art and comparative study. *ACM Computing Surveys 45*, 4 (2013), 1–35.

[65] Yang, J., and Leskovec, J. Defining and evaluating network communities based on ground-truth. In *2012 IEEE 12th International Conference on Data Mining* (2012), pp. 745–754.

[66] Yang, J., and Leskovec, J. Overlapping communities explain core-periphery organization of networks. *Proceedings of the IEEE 102*, 12 (2014), 1892–1902.